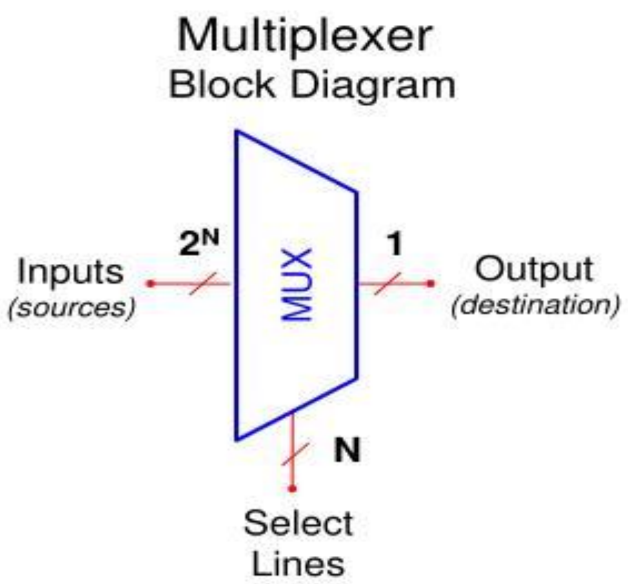
2.4 Multiplexer,Decoder

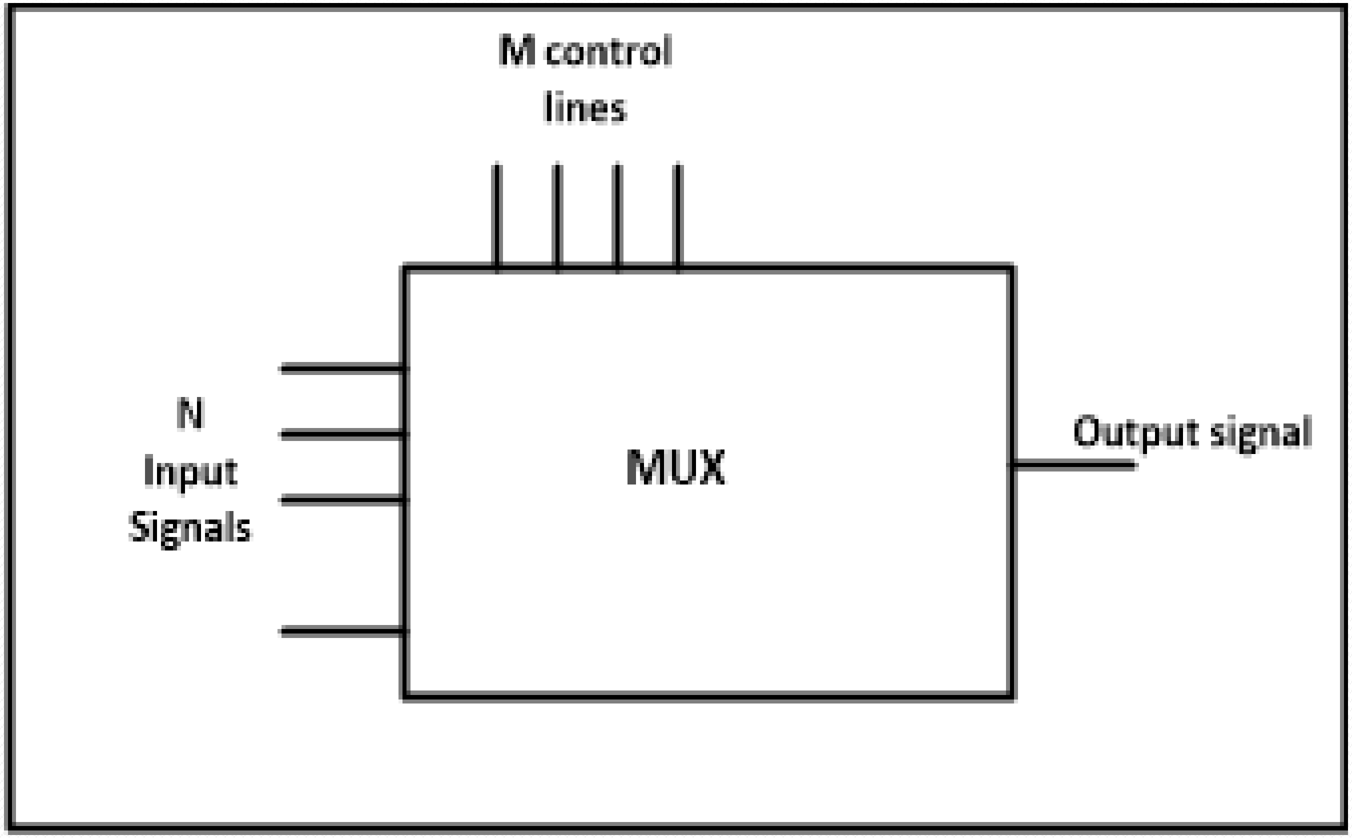
Multiplexer

What is a Multiplexer (MUX)?

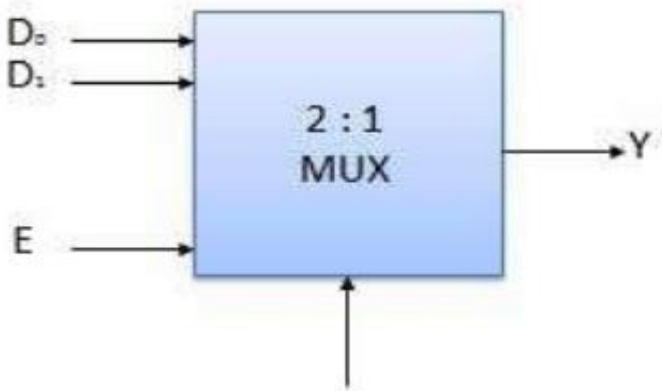
- A MUX is a digital switch that has multiple inputs (sources) and a single output (destination).
- The select lines determine which input is connected to the output.
- MUX Types
  - → 2-to-1 (1 select line)
  - → 4-to-1 (2 select lines)
  - → 8-to-1 (3 select lines)
  - → 16-to-1 (4 select lines)



2


M control
lines
N
Input
Signals
MUX
Output signal

- Multiplexers come in multiple variations
- 2 : 1 multiplexer
- 4 : 1 multiplexer
- 16 : 1 multiplexer
- 32 : 1 multiplexer

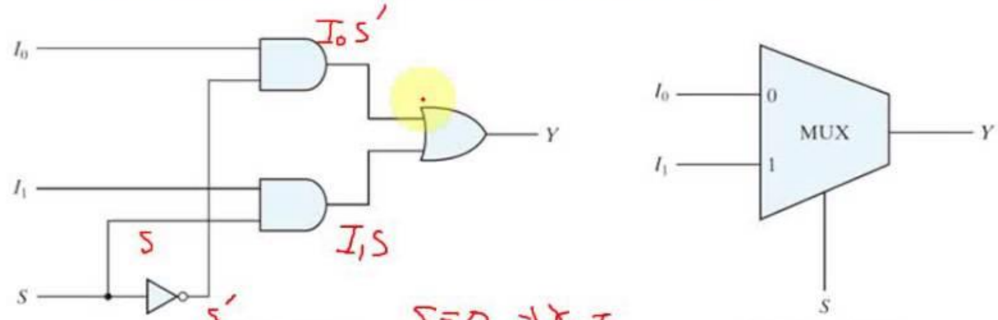


Block Diagram

Enable	Select	Output
E	S	Y
0	x	0
1	0	D_0
1	1	D_1

x = Don't care

2-to-1-line multiplexer

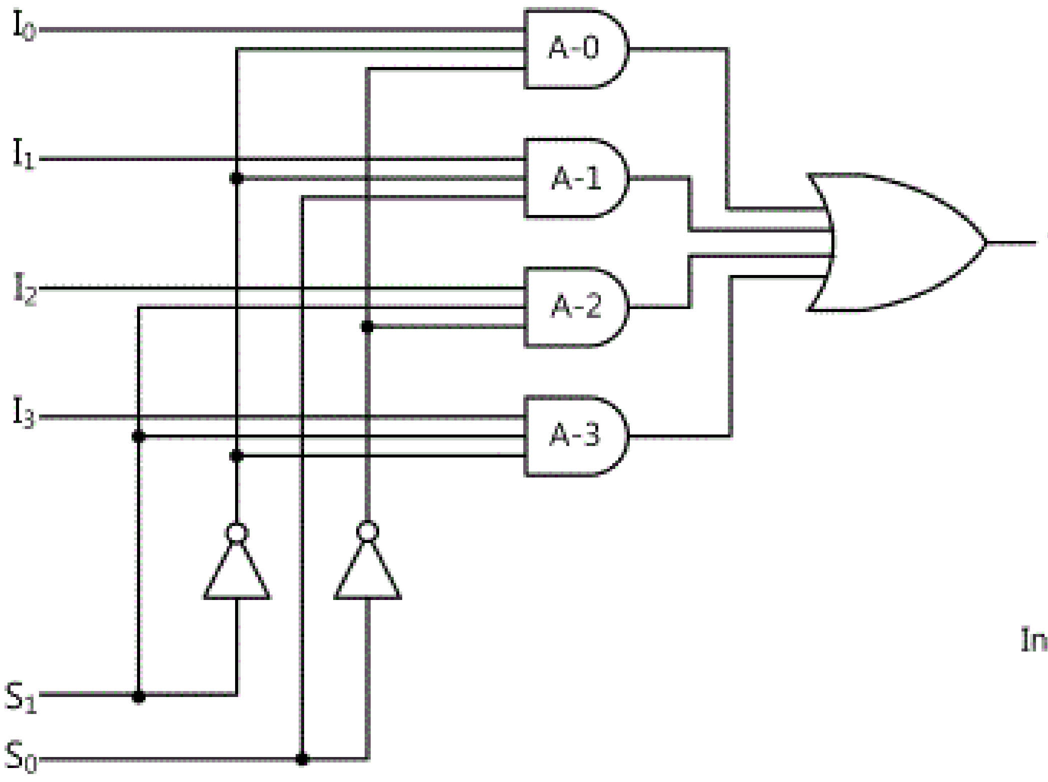


(a) Logic diagram (b) Block diagram

$S=0 \Rightarrow Y=I_0$

$S=1 \Rightarrow Y=I_1$

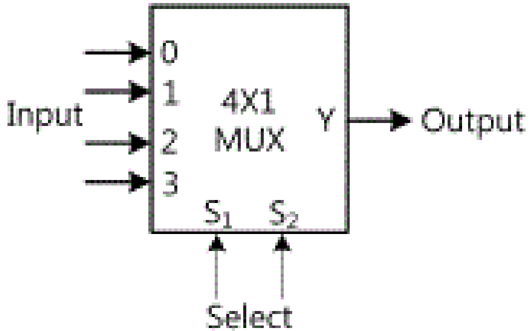
4:1 Multiplexer



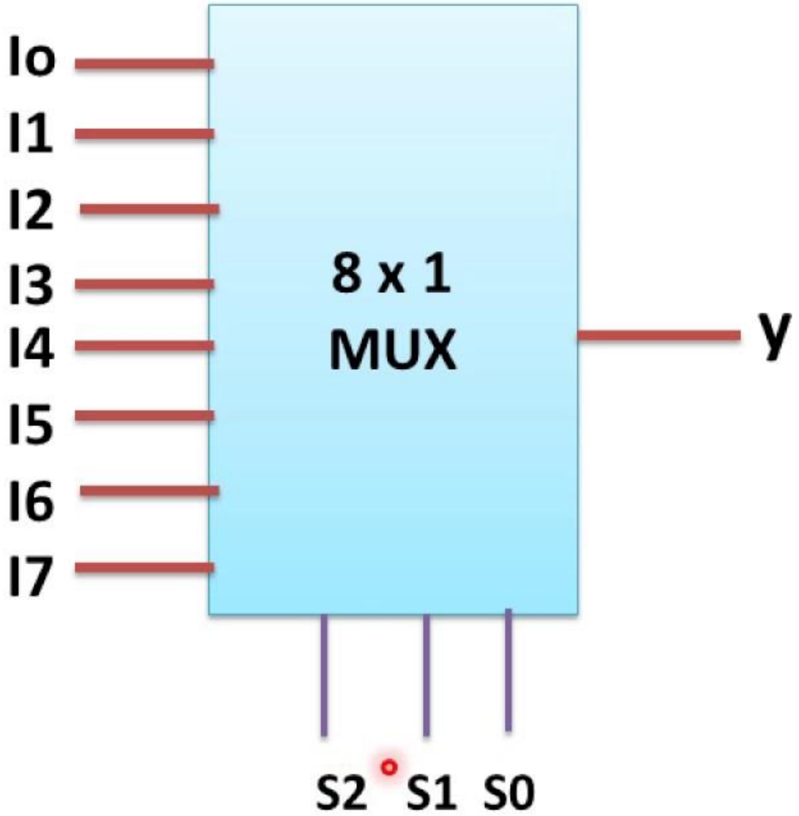
(a) Logic Diagram

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

(b) Function Table

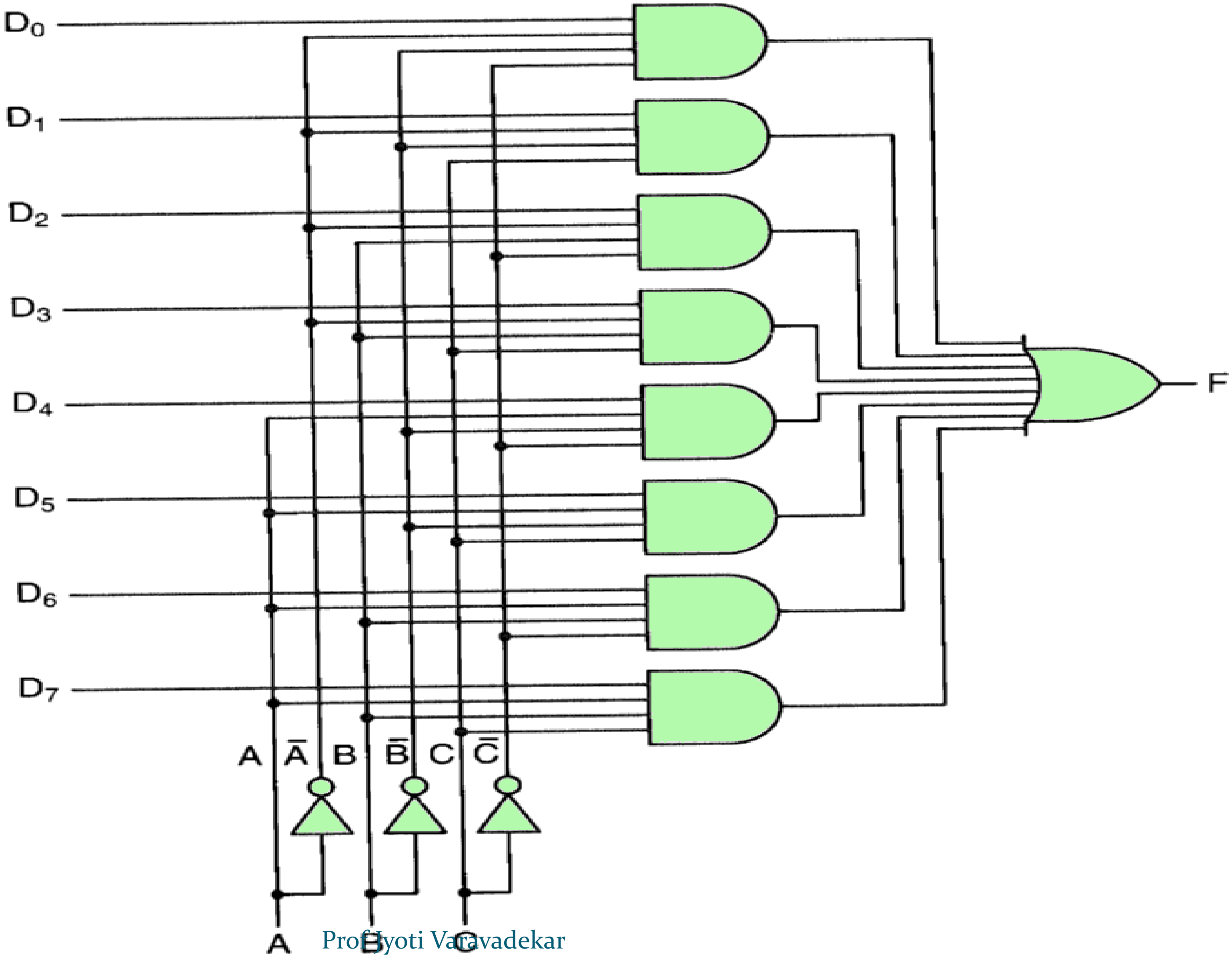


(c) Block Diagram

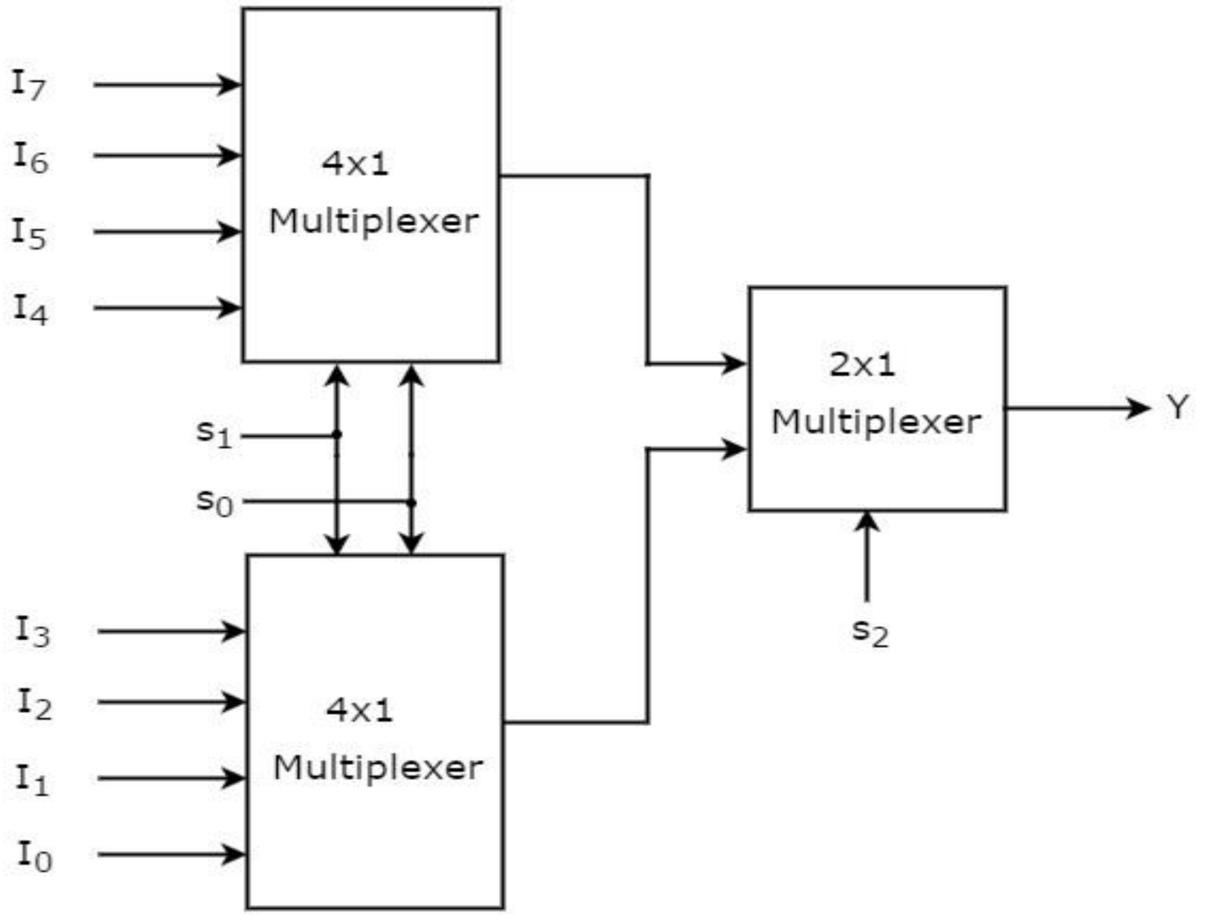


S2	S1	S0	Y
0	0	0	Io
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

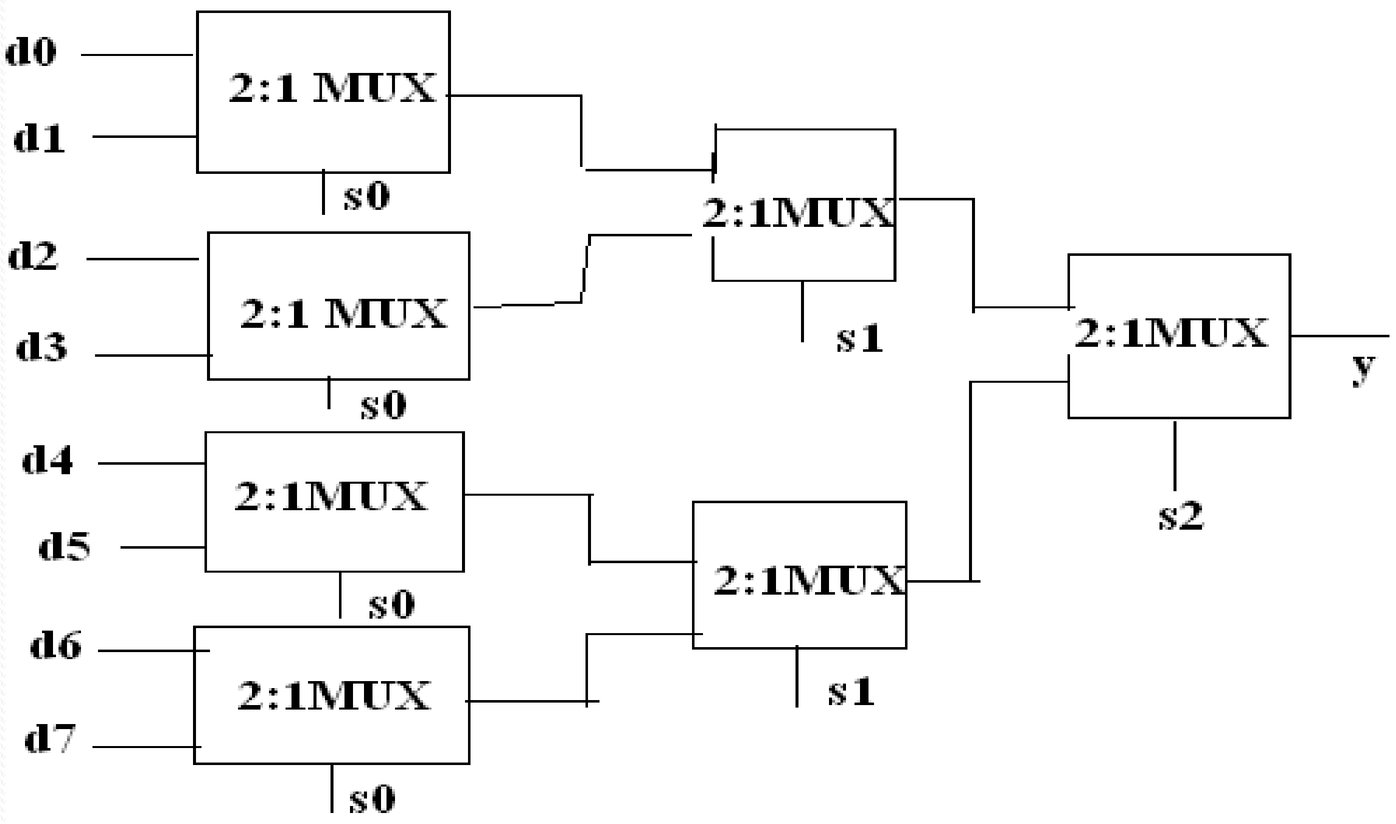
8:1 mux gate level implementation



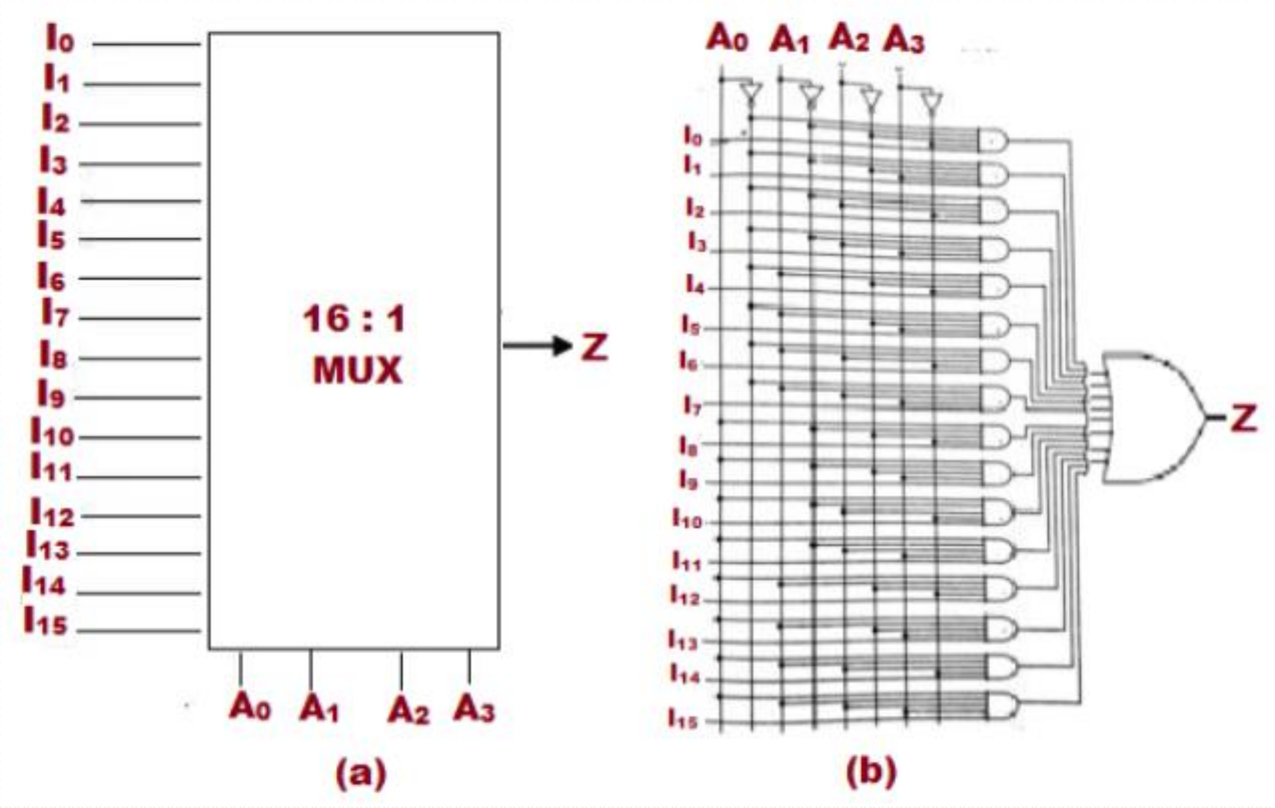
8:1 mux using 4:1 mux



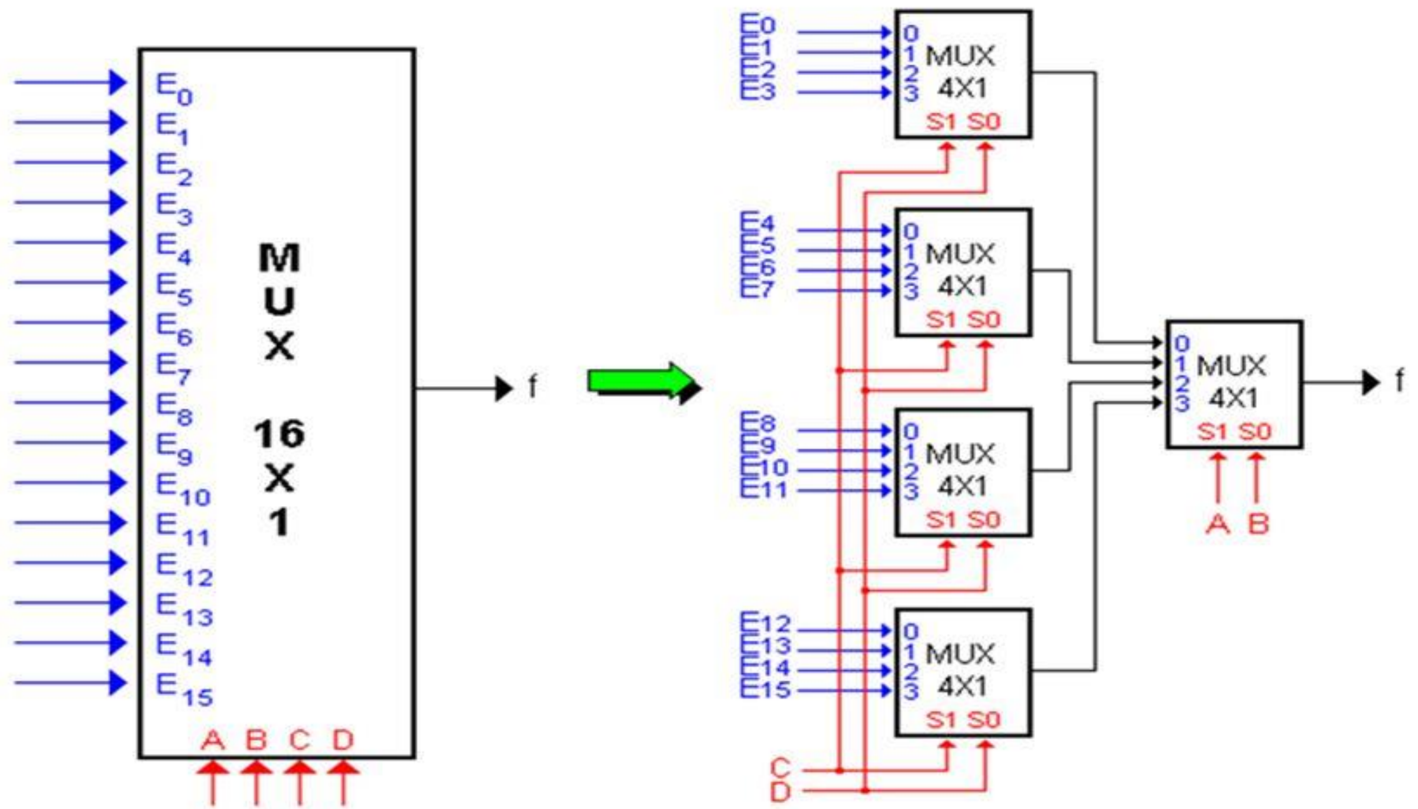
8:1 mux using 2:1 mux



16:1 Mux


I0
I1
I2
I3
I4
I5
I6
I7
I8
I9
I10
I11
I12
I13
I14
I15
16 : 1
MUX
Z
A0
A1
A2
A3
(a)
A0 A1 A2 A3
Z
(b)

16-to-1 MUX

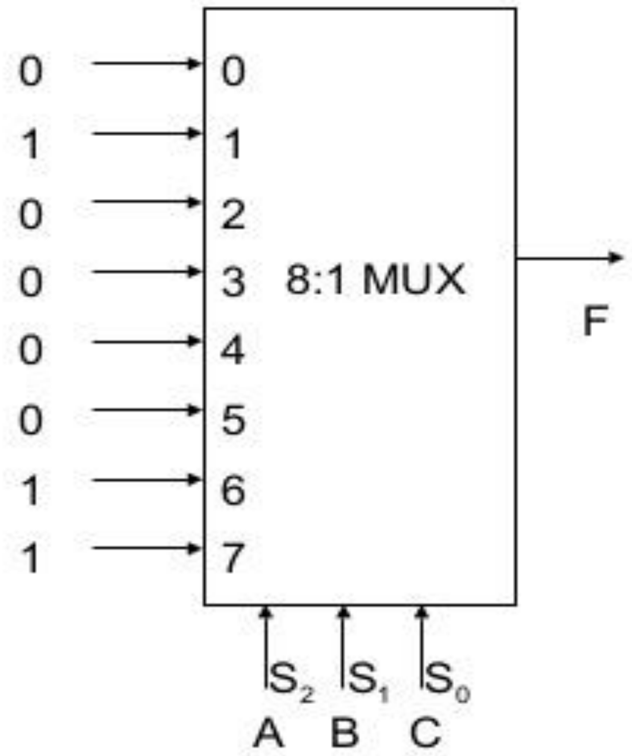


E0
E1
E2
E3
E4
E5
E6
E7
E8
E9
E10
E11
E12
E13
E14
E15
M U X 16 X 1
A B C D
f
0 1 2 3
MUX 4X1
S1 S0
A B
C
D

Implementation Of Logic Functions using Multiplexer

f(a, b, c) = a'b'c + ab

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

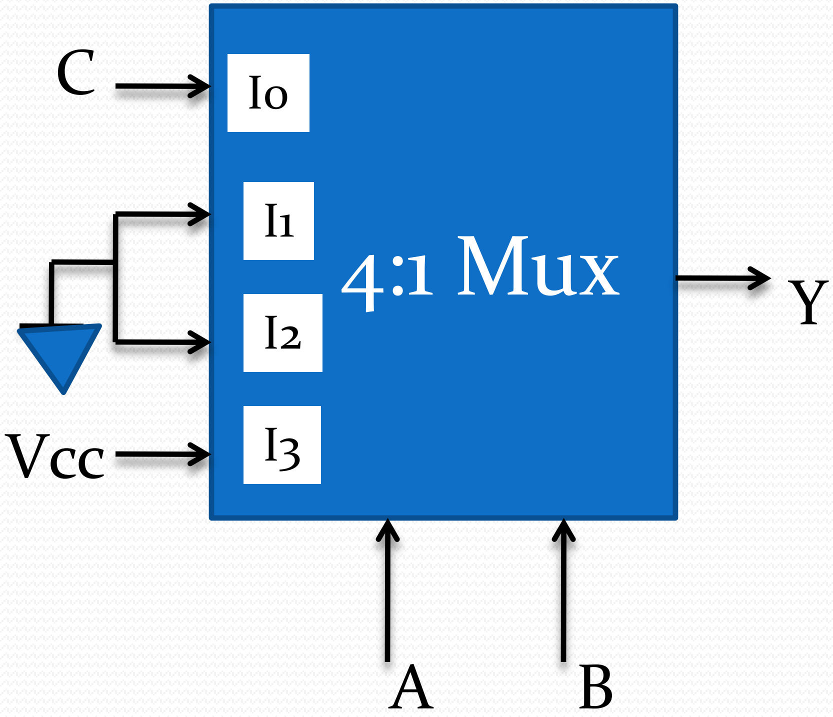


Example of 3 variable function realization using one 4:1 Mux:Method 1

- $F = \sum m(1,6,7)$

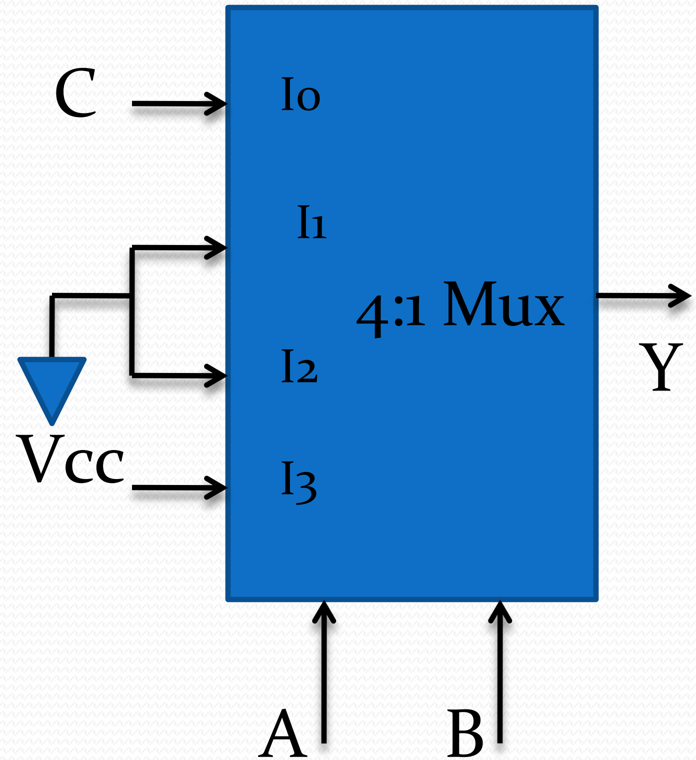
Truth table

A	B	C	Y	
0	0	0	0	C
0	0	1	1	
0	1	0	0	**0**
0	1	1	0	
1	0	0	0	**0**
1	0	1	0	
1	1	0	1	**1**
1	1	1	1	

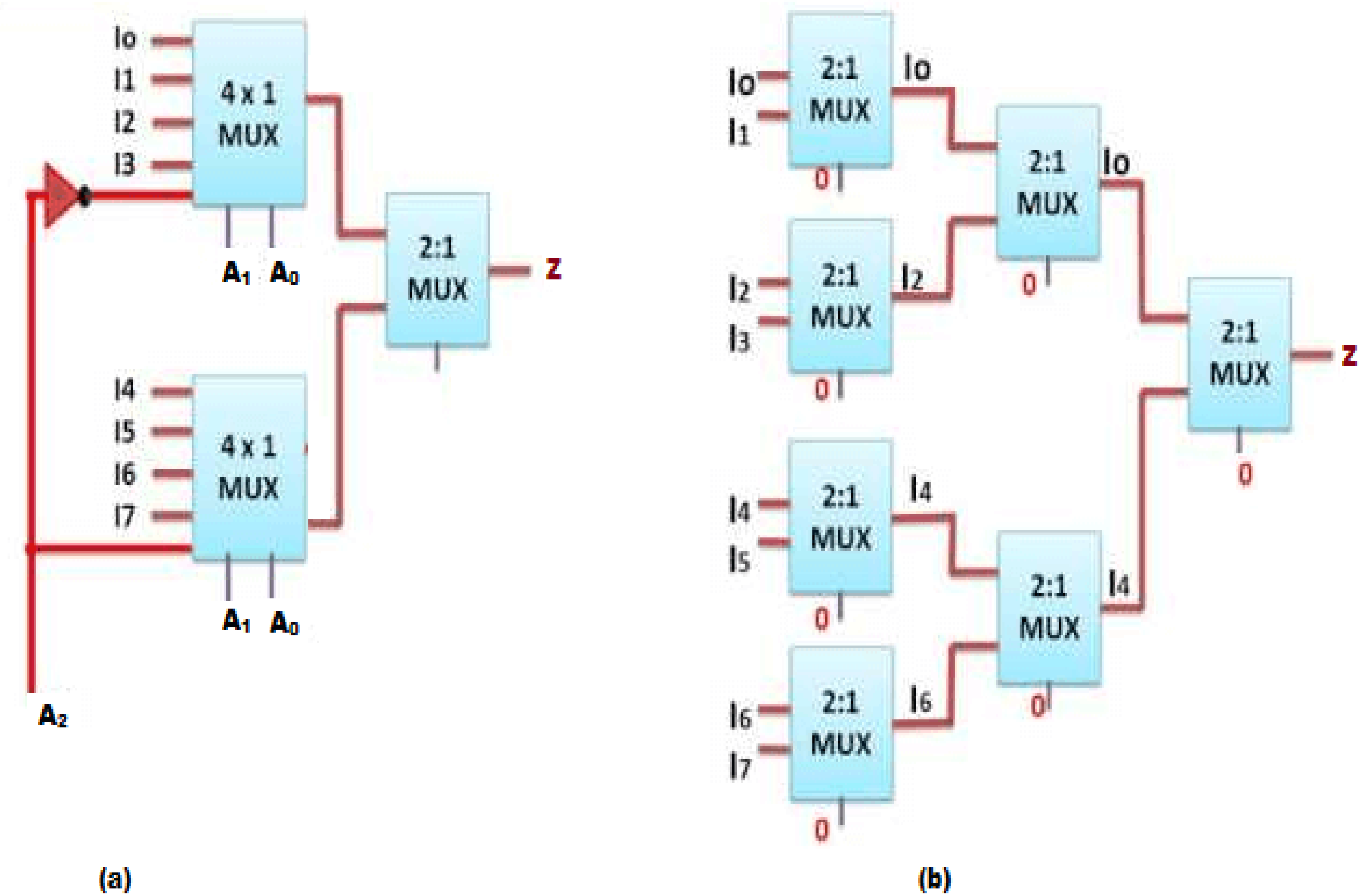


Example of 3 variable function realization using one 4:1 Mux: Method 2

	I_0	I_1	I_2	I_3
C'	0	2	4	6
C	1	3	5	7
	C	0	0	1

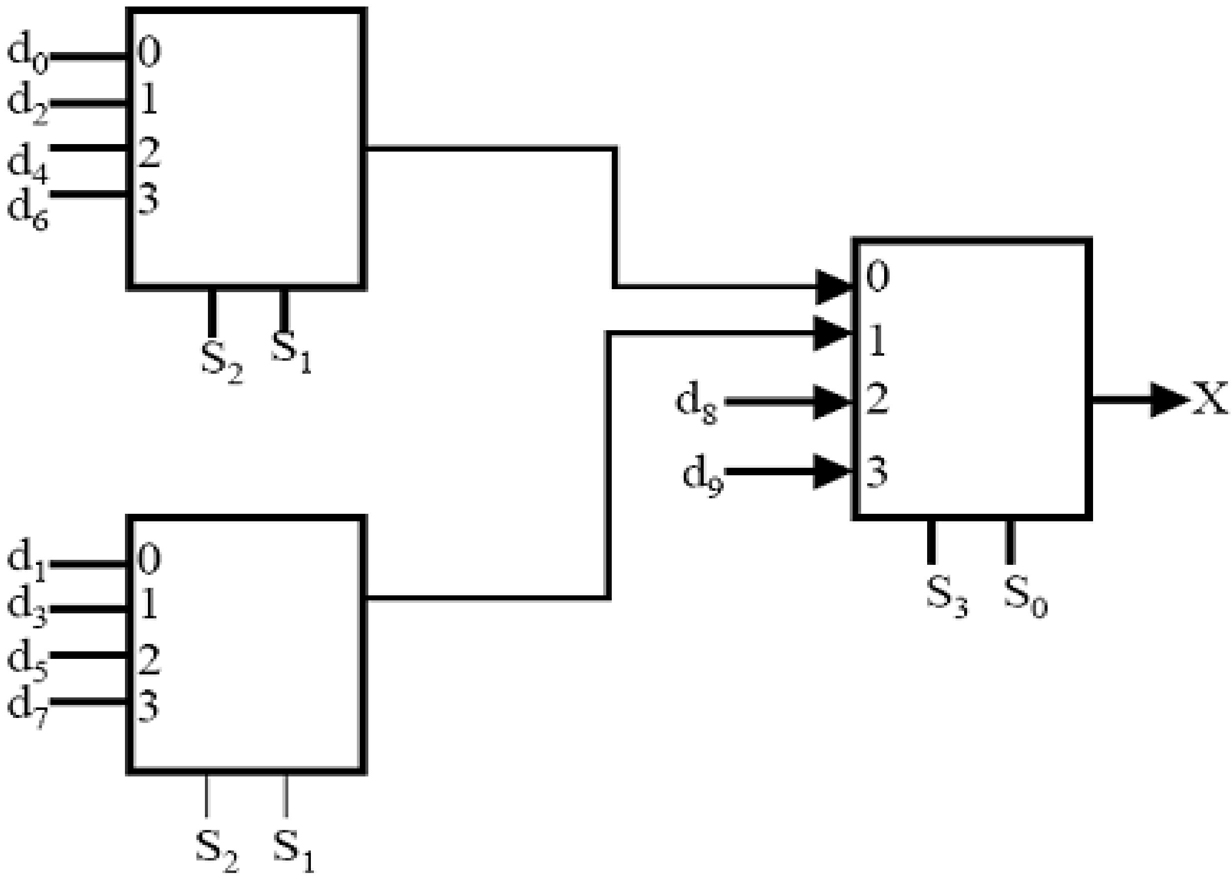


Cascading multiplexer



Cascading multiplexer

Implementation using 4:1 muxes.

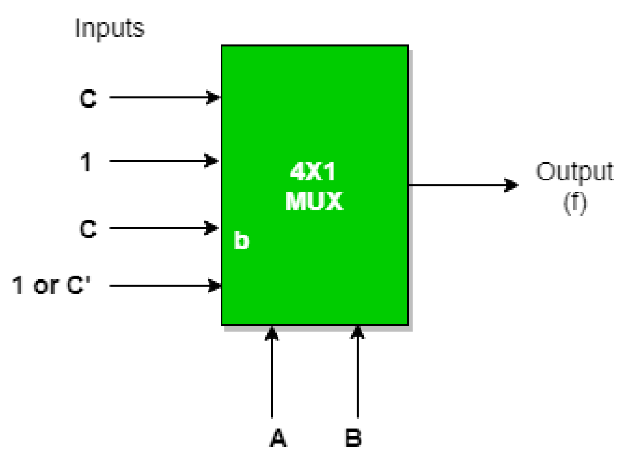


- **An example to implement a boolean function if minimal and don’t care terms are given using MUX.**
  f (A, B, C) = Σ (1, 2, 3, 5, 6) with don’t care (7) using
  4 : 1 MUX using as
  a) AB as select : Expanding the minterms to its boolean form and will see its 0 or 1 value in Cth place so that they can be placed in that manner.

AB as select

A	B	C	f
0	0	0	C’
0	0	1	C
0	1	0	C’
0	1	1	C
1	0	0	C’
1	0	1	C
1	1	0	C’
1	1	1	C

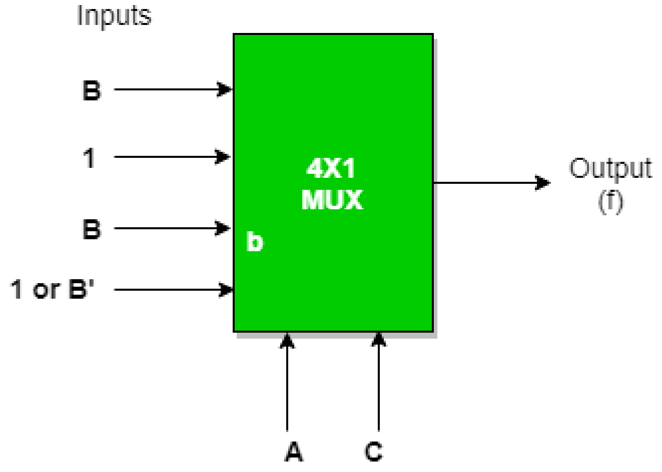
	I0	I1	I2	I3
C’	0	2	4	6
C	1	3	5	7 is don’t care (can consider or not)
	C	1	C	1 (in case 7 is considered or C’)



AC as select

A	B	C	f
0	0	0	B’
0	0	1	B’
0	1	0	B
0	1	1	B
1	0	0	B’
1	0	1	B’
1	1	0	B
1	1	1	B

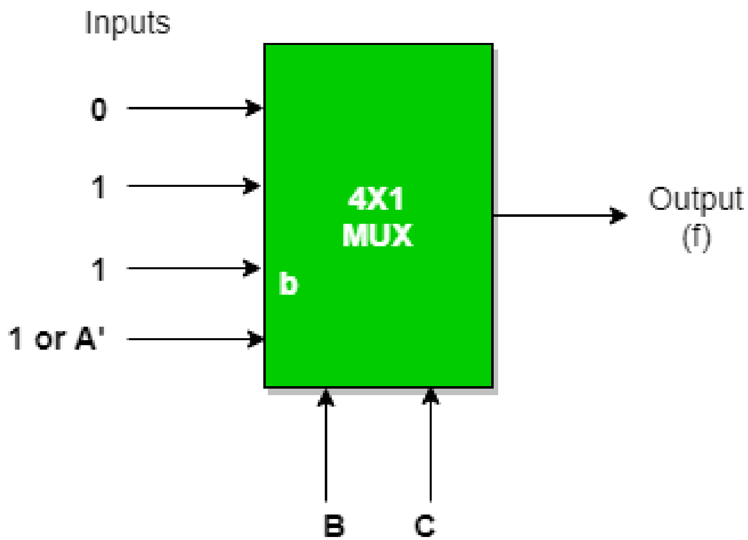
	I0	I1	I2	I3
B’	0	1	4	5
B	2	3	6	7 is don’t care (can consider or not)
	B	1	B	1 (in case 7 is considered) or B’



BC as select

A	B	C	f
0	0	0	A’
0	0	1	A’
0	1	0	A’
0	1	1	A’
1	0	0	A
1	0	1	A
1	1	0	A
1	1	1	A

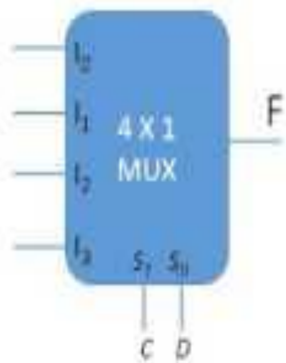
	I0	I1	I2	I3
A’	0	1	2	3
A	4	5	6	7 is don’t care (can considered or not)
	0	1	1	1 (in case 7 is considered) or A’



Implementation using Multiplexer

$F(A, B, C, D) = \sum(1, 3, 4, 11, 12, 13, 14, 15)$ using 4 x 1 Multiplexer with A, B as input variables and C, D as selection variables

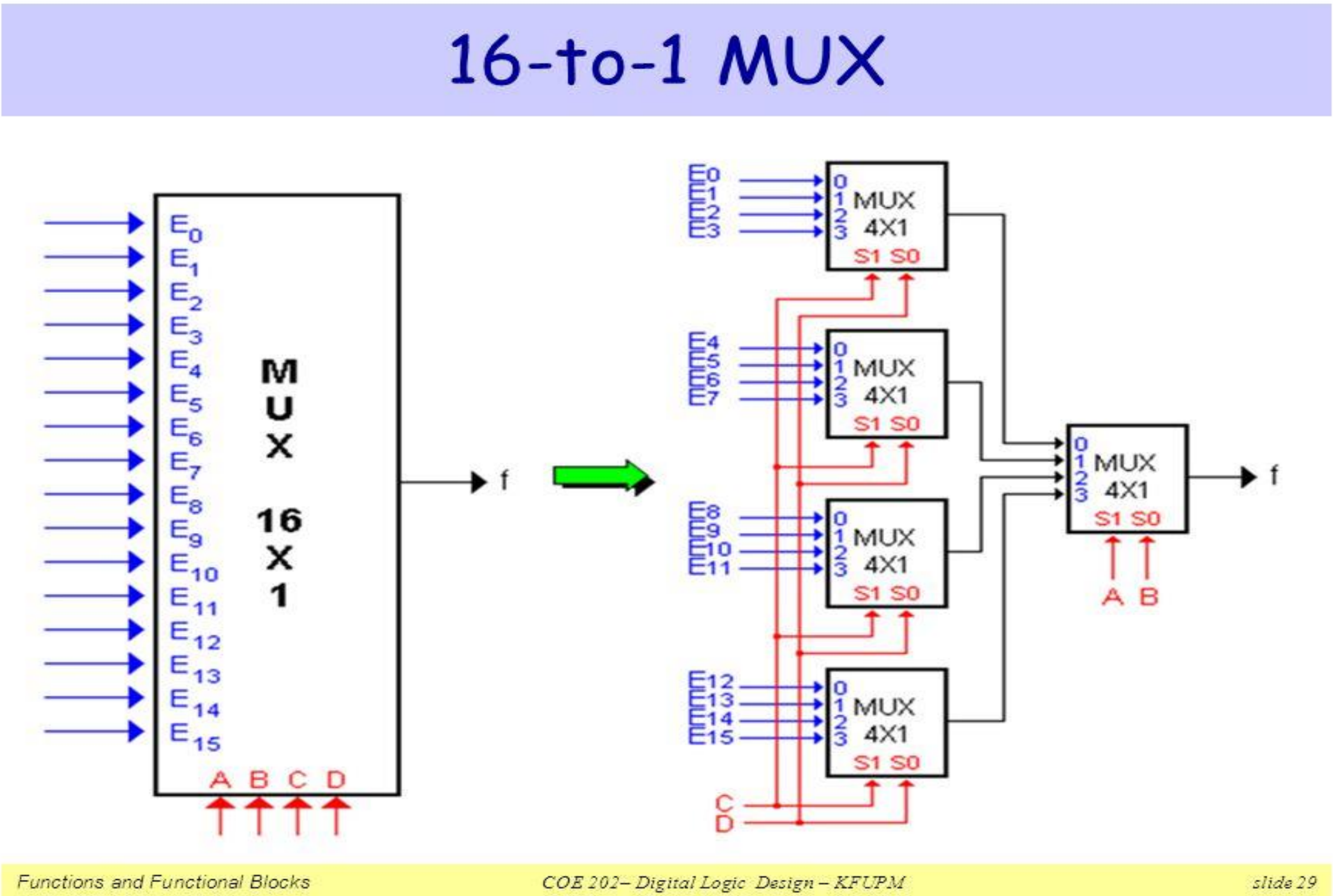
	I_0	I_1	I_2	I_3
A'B'	0	1	2	3
A'B	4	5	6	7
AB'	8	9	10	11
AB	12	13	14	15



A	B	C	D	
0	0	0	0	0
0	0	0	1	1
0	0	1	0	2
0	0	1	1	3
0	1	0	0	4
0	1	0	1	5
0	1	1	0	6
0	1	1	1	7
1	0	0	0	8
1	0	0	1	9
1	0	1	0	10
1	0	1	1	11
1	1	0	0	12
1	1	0	1	13
1	1	1	0	14
1	1	1	1	15

Dr. V. Saritha

16:1 Multiplexer



Advantages of Mux

- Increases the amount of data that can be sent over the network within a certain amount of time and bandwidth .

- Cost Saving.

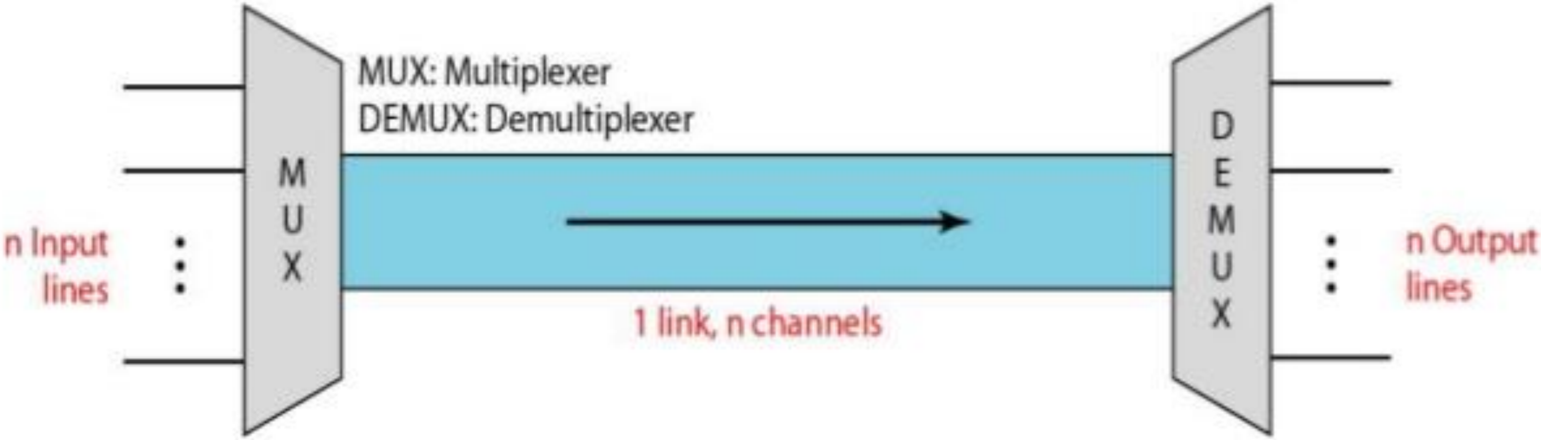
- Larger multiplexers can be constructed by using smaller multiplexers by chaining them together

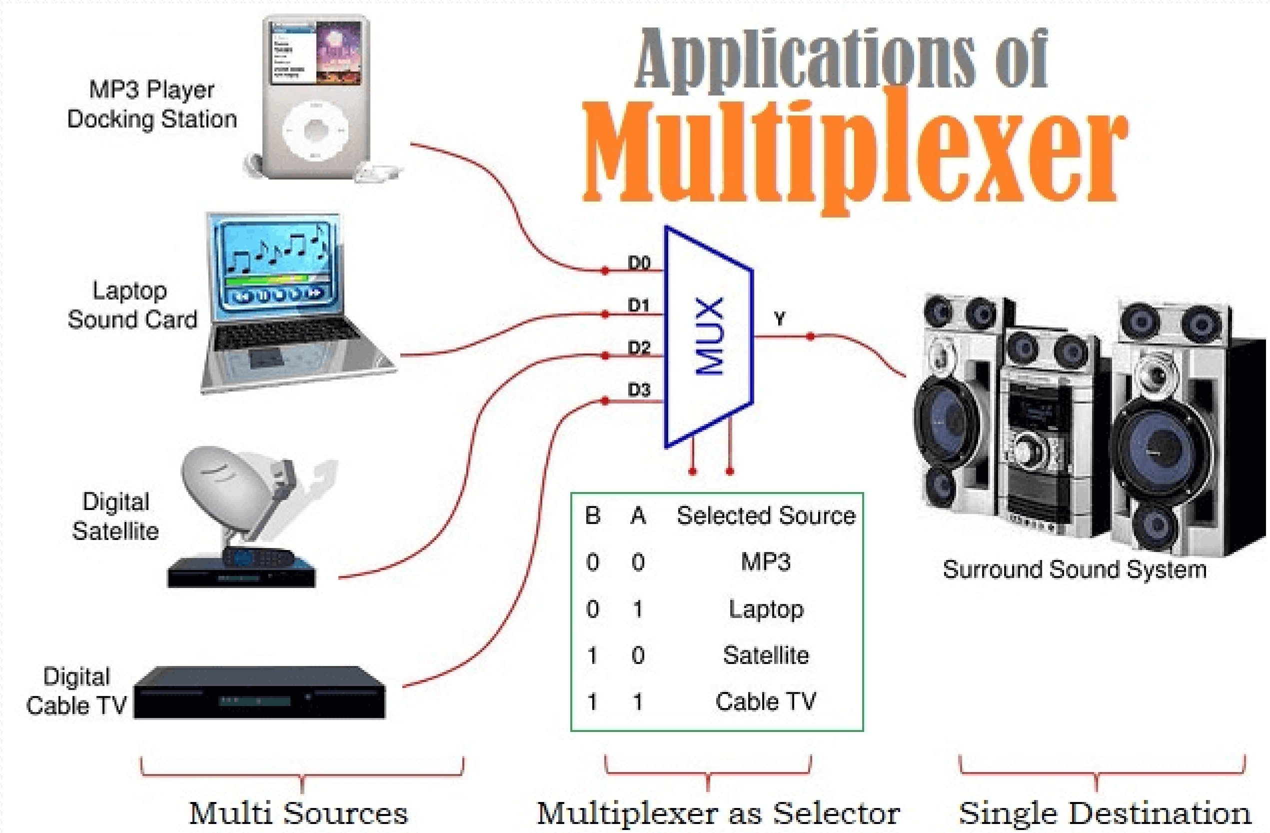
Applications of Multiplexer

- Multiplexer are used in various fields where multiple data need to be transmitted using a single line. Following are some of the applications of multiplexers
- Communication system
- Telephone network
- Computer memory
- Transmission from the computer system of a satellite

Multiplexer and De-Multiplexer

- A multiplexer is a circuit that accept many input but give only one output. A de-multiplexer function exactly in the reverse of a multiplexer, that is a de-multiplexer accepts only one input and gives many outputs. Generally multiplexer and de-multiplexer are used together, because of the communication systems are bi directional.

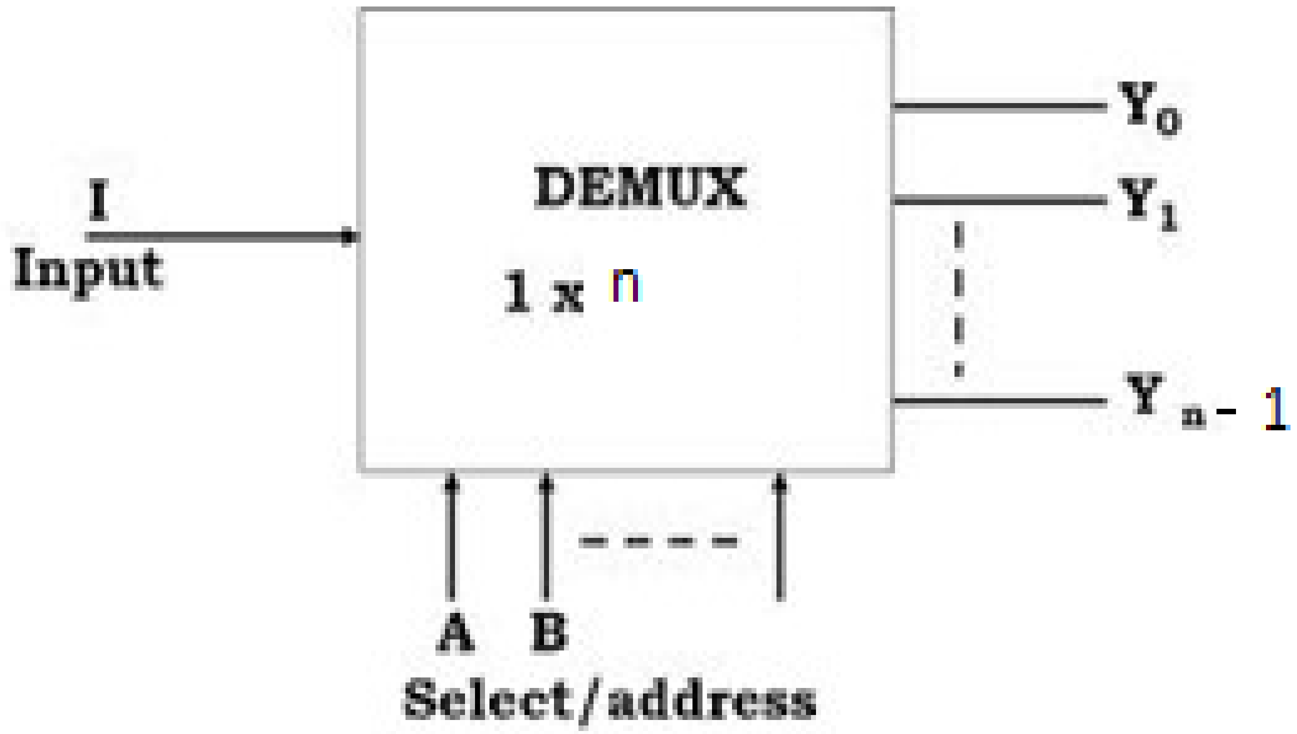



Applications of
Multiplexer
MP3 Player
Docking Station
Laptop
Sound Card
Digital
Satellite
Digital
Cable TV
D0
D1
D2
D3
MUX
Y
Surround Sound System
B A Selected Source
0 0 MP3
0 1 Laptop
1 0 Satellite
1 1 Cable TV
Multi Sources
Multiplexer as Selector
Single Destination

Demultiplexer

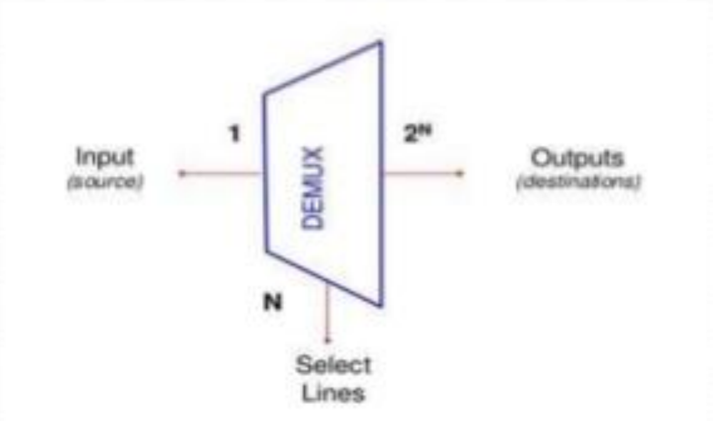
- **Demultiplexers**
- A demultiplexer performs the reverse operation of a multiplexer i.e. it receives one input and distributes it over several outputs.
- It has only one input, n outputs, m select input. At a time only one output line is selected by the select lines and the input is transmitted to the selected output line.
- The demultiplexer takes one single input data line and then switches it to any one of a number of individual output lines one at a time.
- The **demultiplexer** converts a serial data signal at the input to a parallel data at its output lines as shown below.
- A de-multiplexer is equivalent to a single pole multiple way switch.

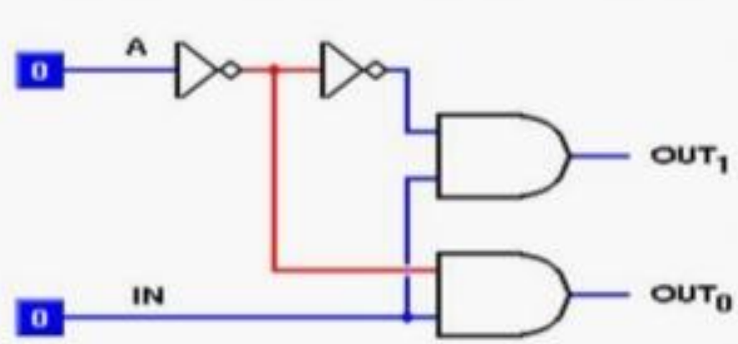
Schematic of Demultiplexer



Demultiplexer

<u>1- to -2 line demultiplexer</u>

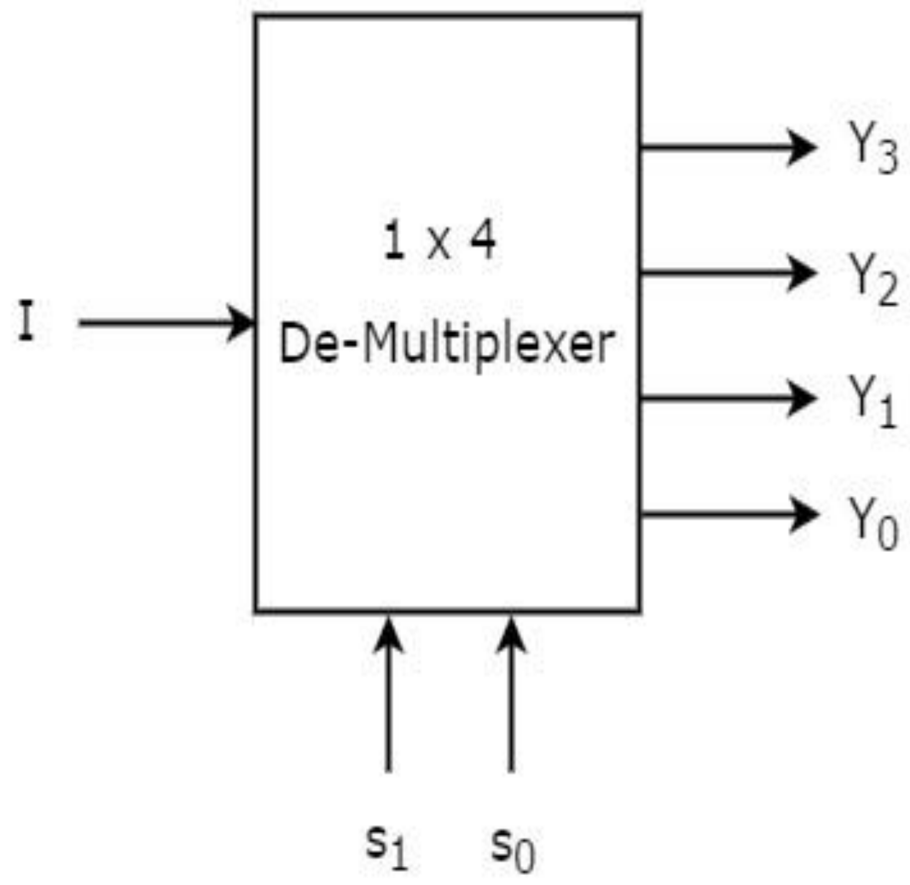




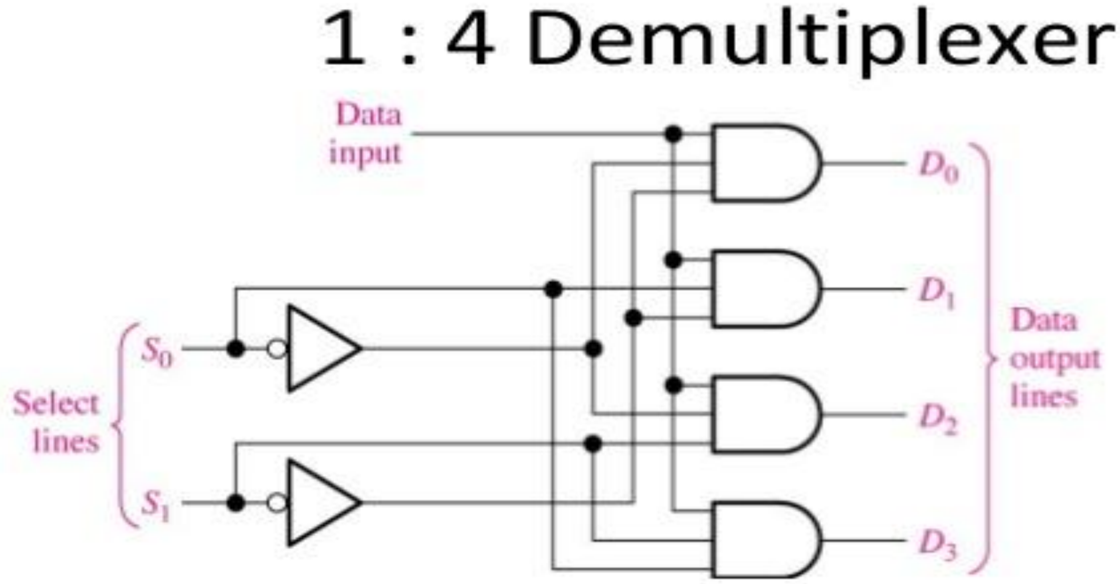
TRUTH TABLE

INPUTS		OUTPUTS	
A	IN	OUT_1	OUT_0
0	0	0	0
0	1	0	1
1	0	0	0
1	1	1	0

1:4 Demultiplexer

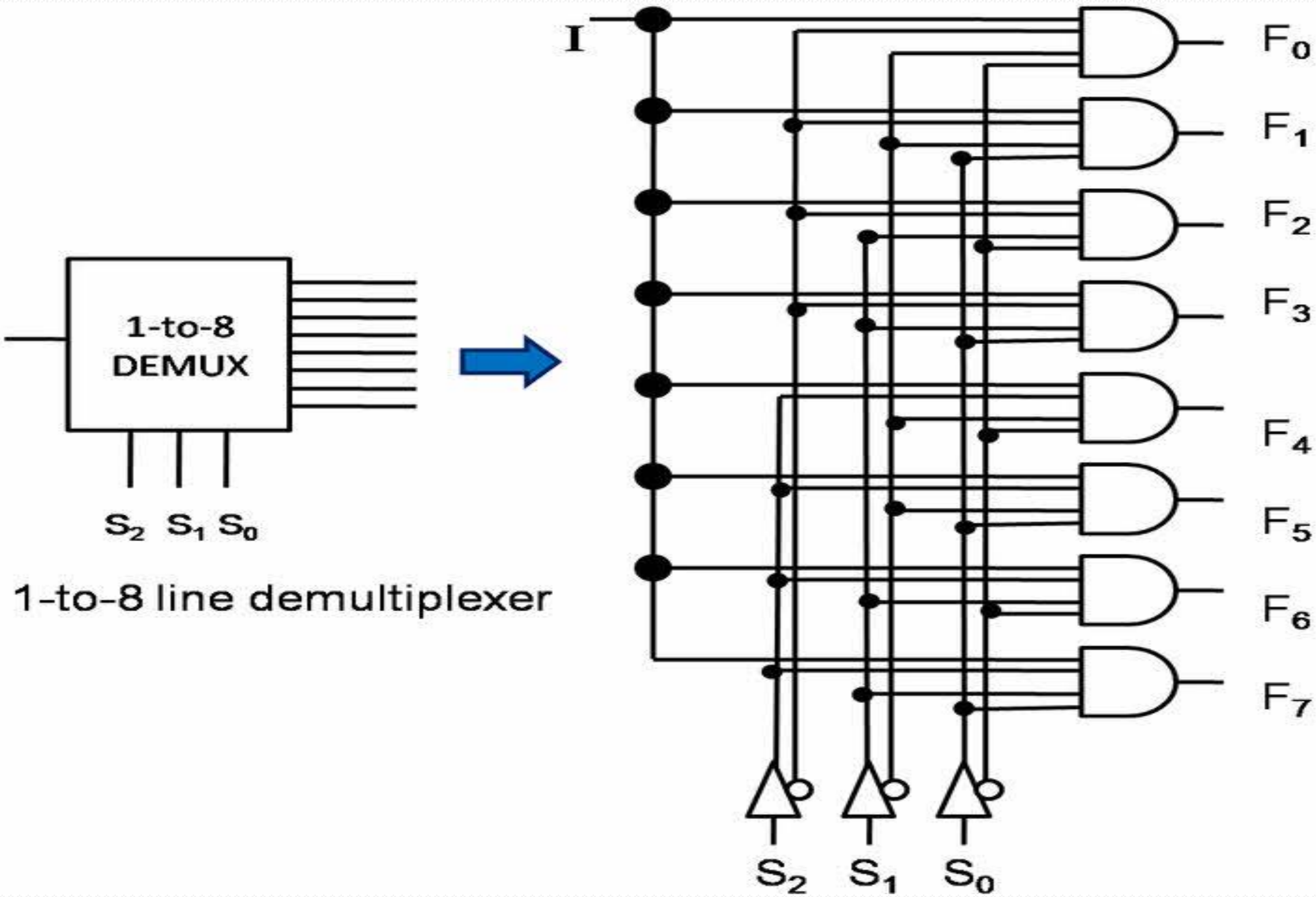


1:4 Demultiplexer gate level circuit



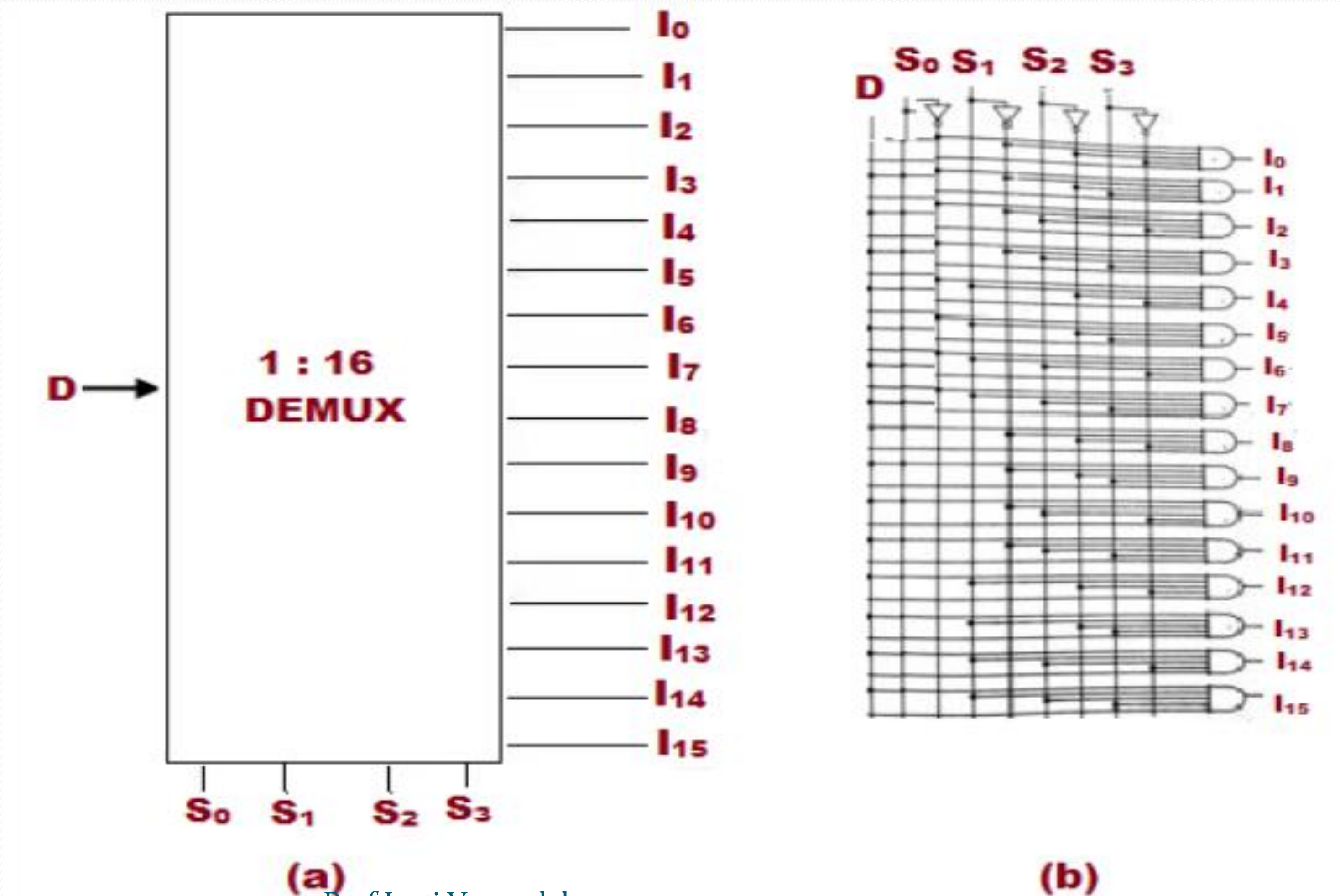
S_0	S_1	D_0	D_1	D_2	D_3
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D

1:8 Demultiplexer

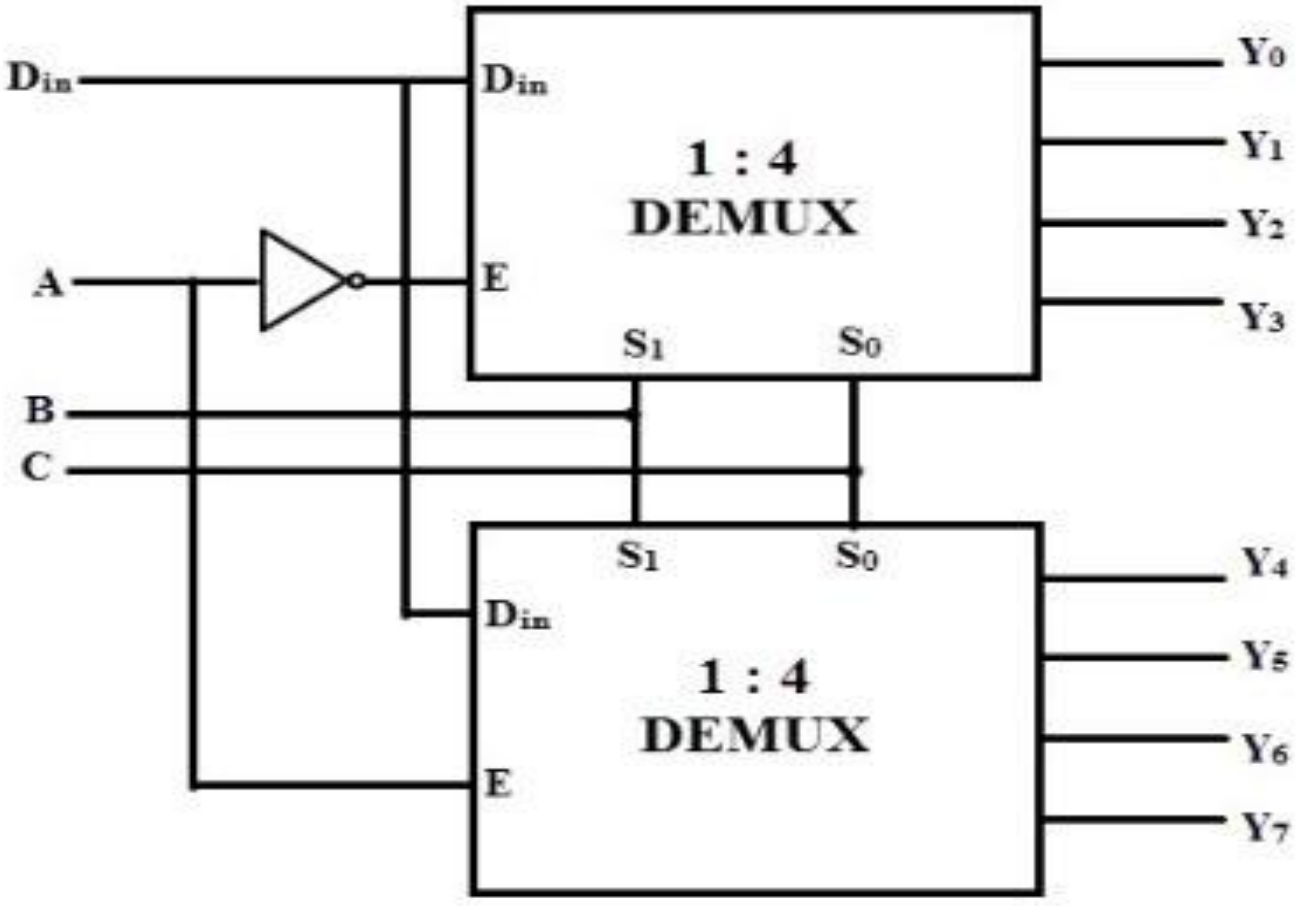


1-to-8 line demultiplexer

1:16 Demultiplexer



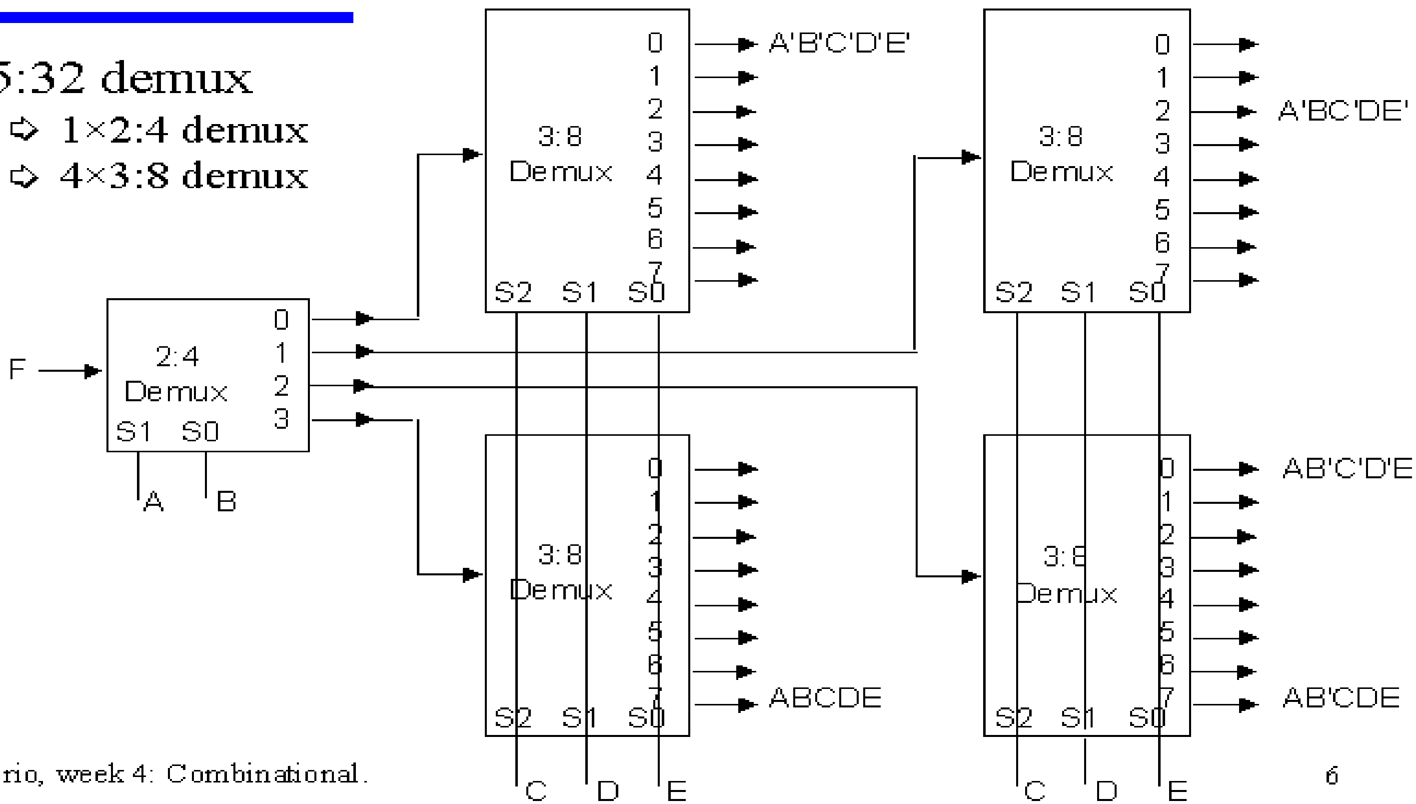
1:8 demux using two 1:4 demux



1:32 demux using cascading

Cascading demultiplexers

- 5:32 demux
  - 1×2:4 demux
  - 4×3:8 demux



C. Diorio, week 4: Combinational. 6

Case 1

- Implement the function f(A,B,C)=m(0,1,3,5) using 1:8 demultiplexer

MSB LSB

Find the number of select lines

General Form of a Demultiplexer is i/p : o/p

$1 : 2^n$

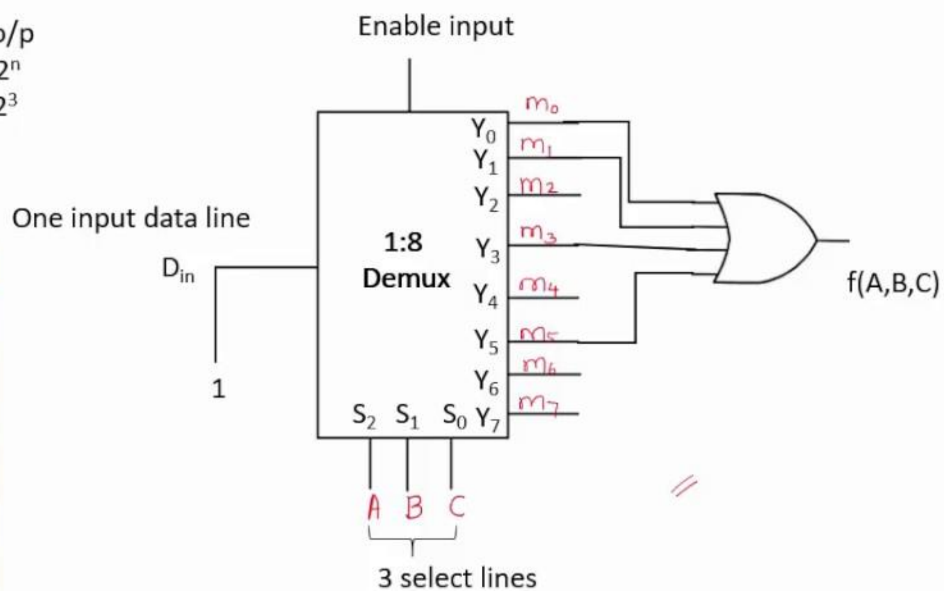
$1 : 2^3$

No of select lines (n) = 3

No of input = 1

No of outputs = 8

A	B	C	f(A,B,C)
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

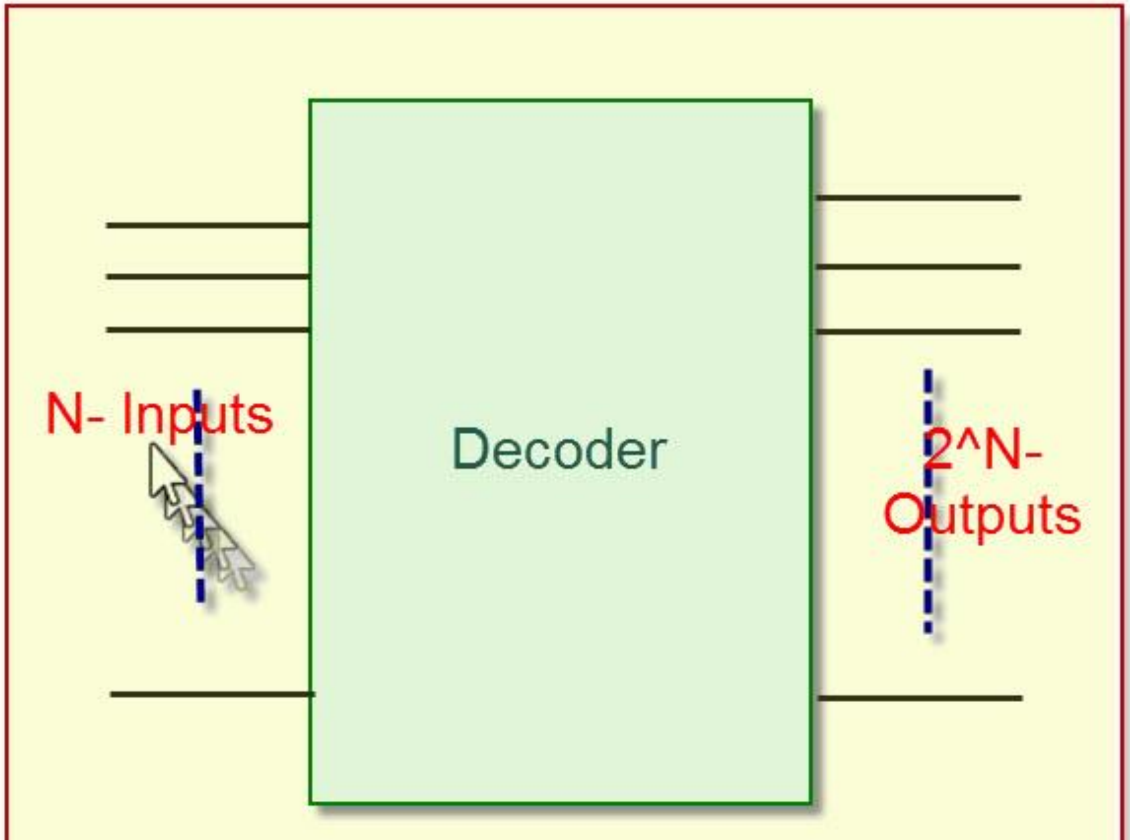


Applications of De-Multiplexer

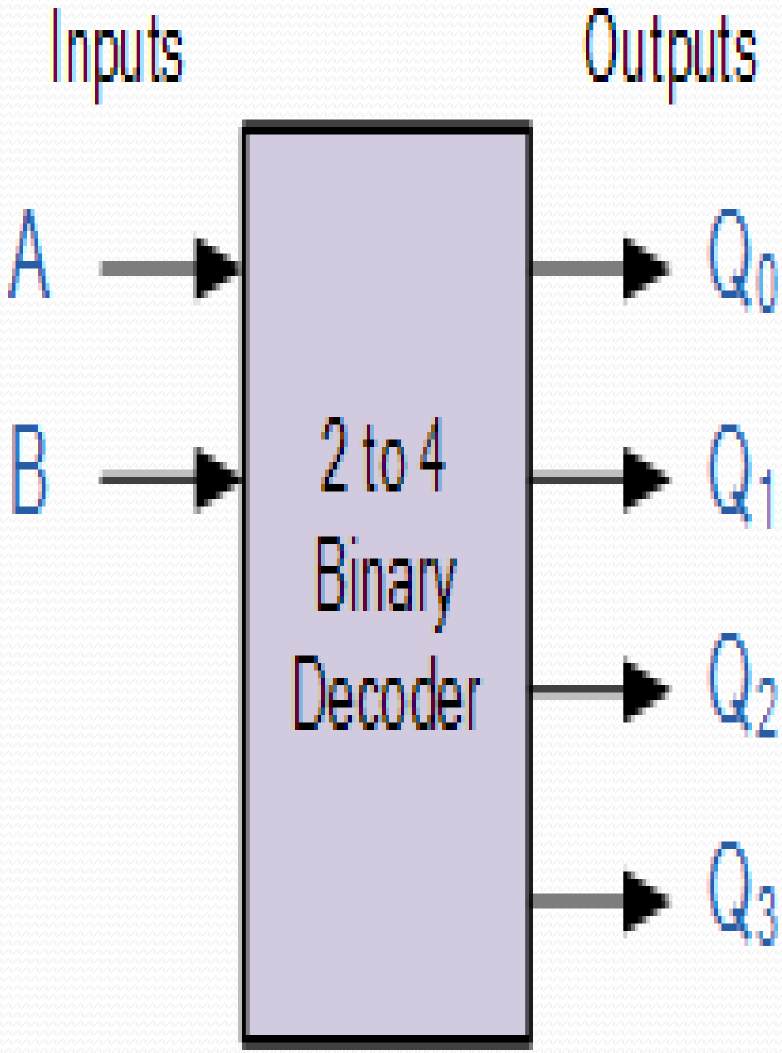
- De-multiplexer is used to connect a single source to multiple destinations. The main application area of de-multiplexer is communication system where multiplexer are used.
- Communication System
- ALU (Arithmetic Logic Unit)
- Serial to parallel converter

Decoder

It is a combinational circuit that converts n bit binary information at its input into max of 2^n of output lines



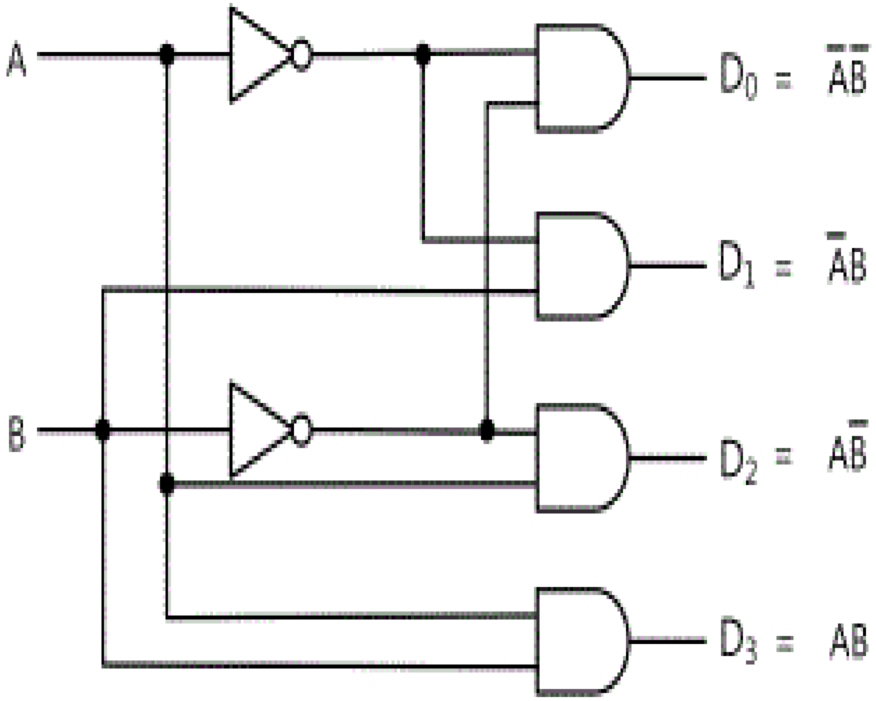
2-4 Decoder



Truth Table

A	B	Q_0	Q_1	Q_2	Q_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Gate level 2- 4 line Decoder



(a) Circuit of 2x4 Decoder

Input		Output			
A	B	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

(b) Truth Table of 2x4Decoder

3-8 line Decoder

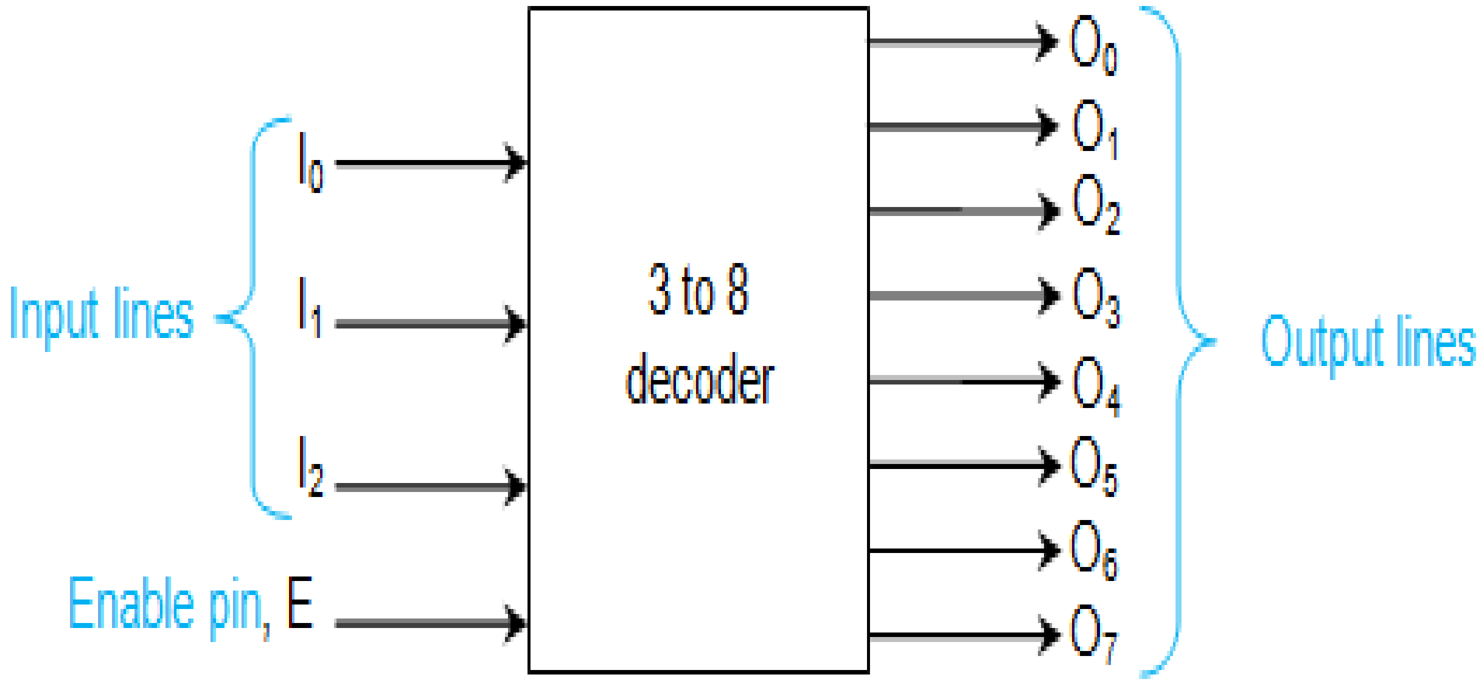
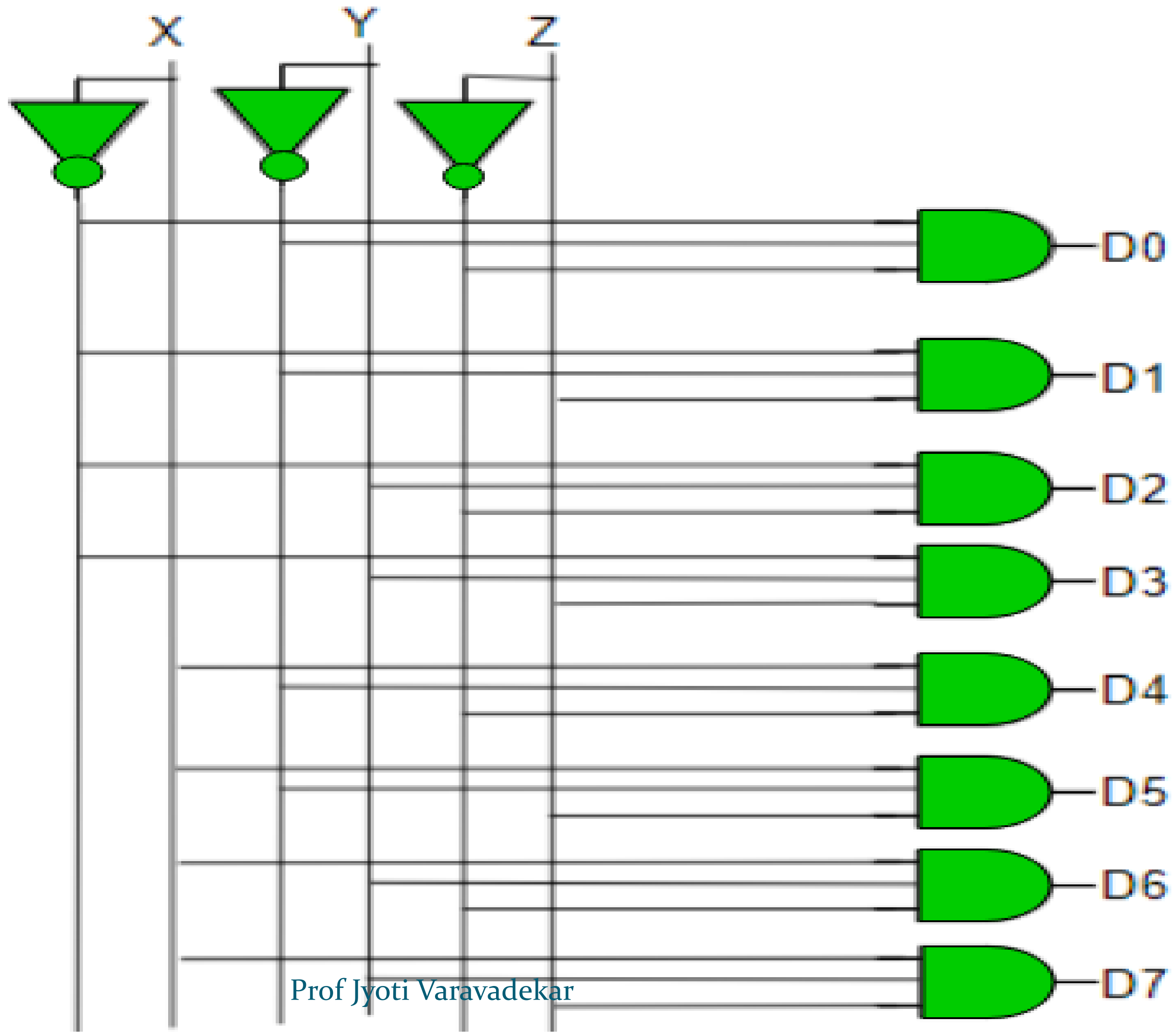


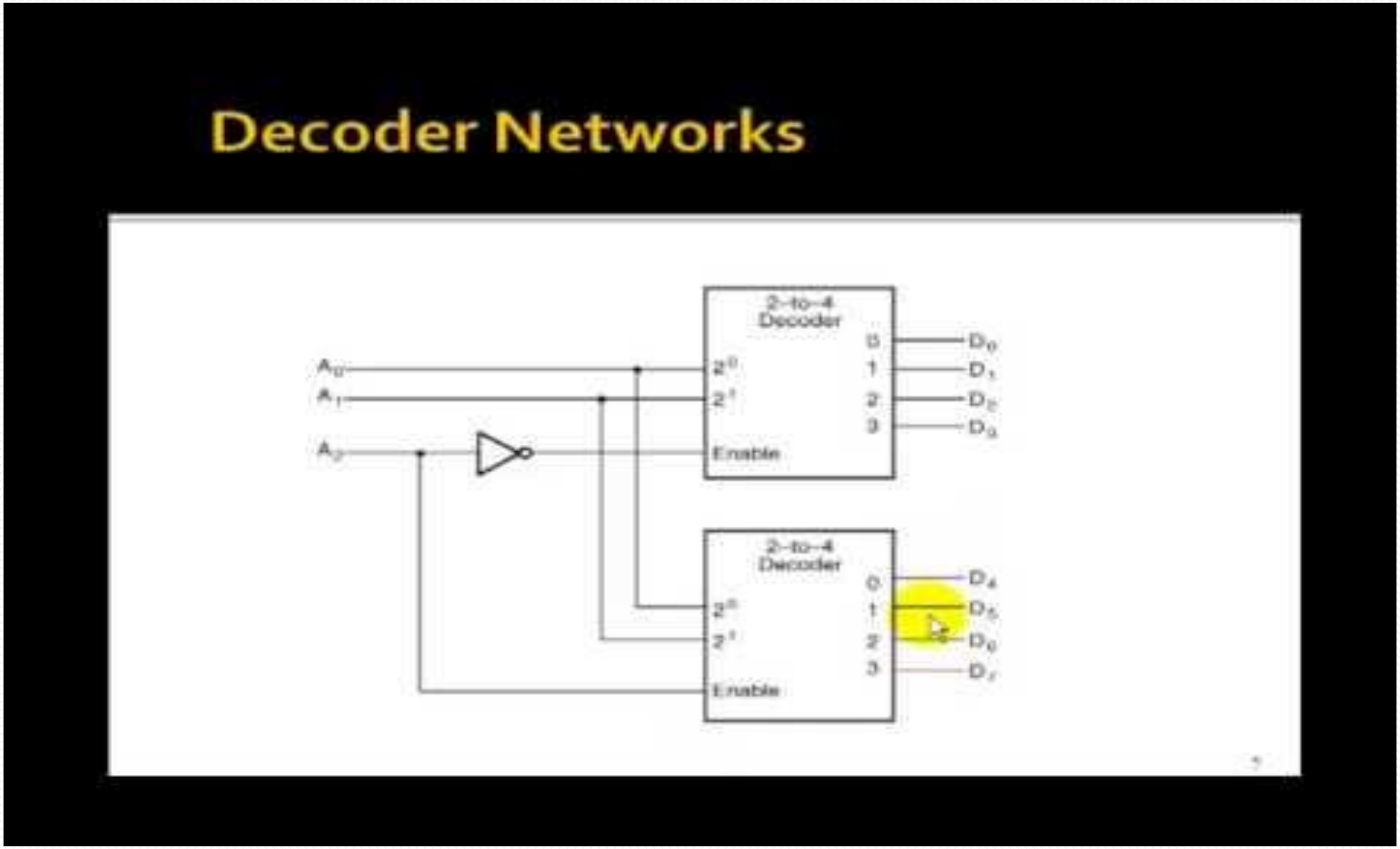
Figure 1 3 to 8 binary decoder

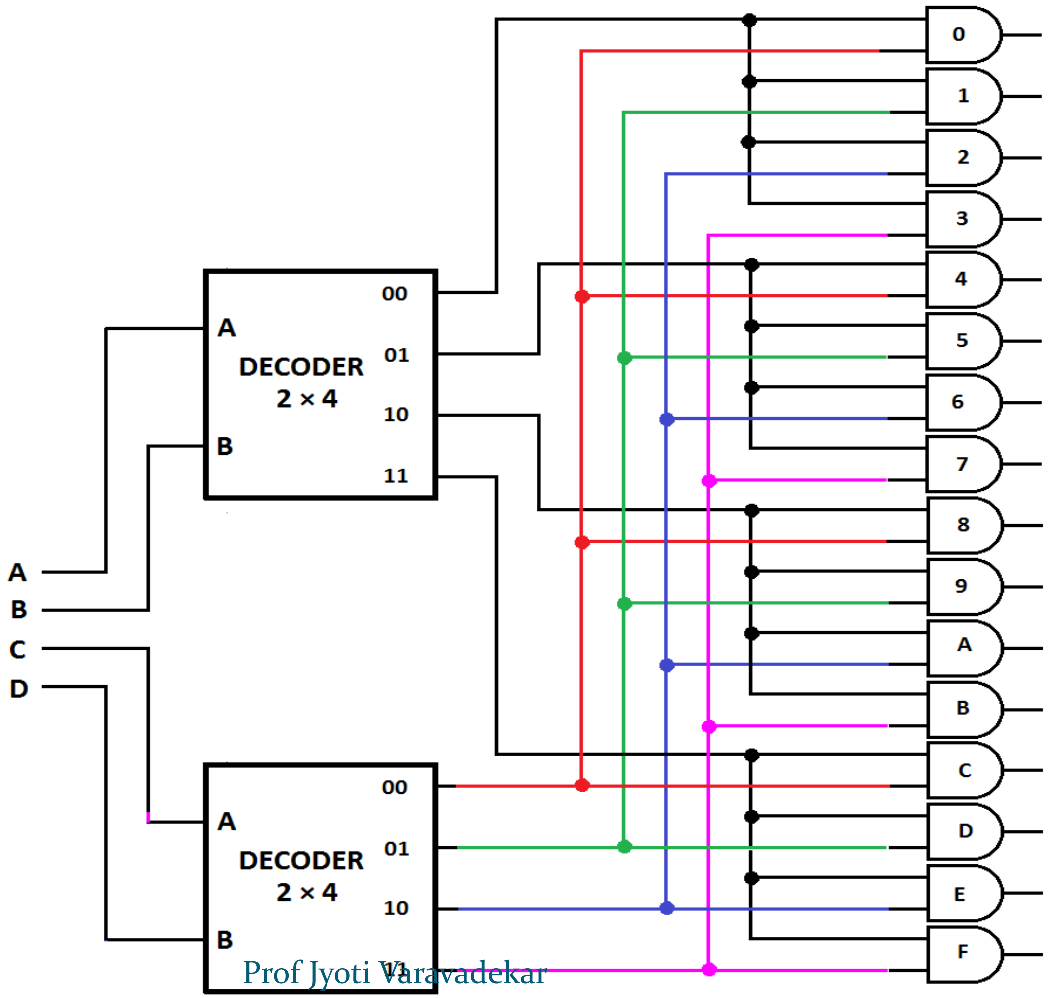
3-8 gate level decoder implementation



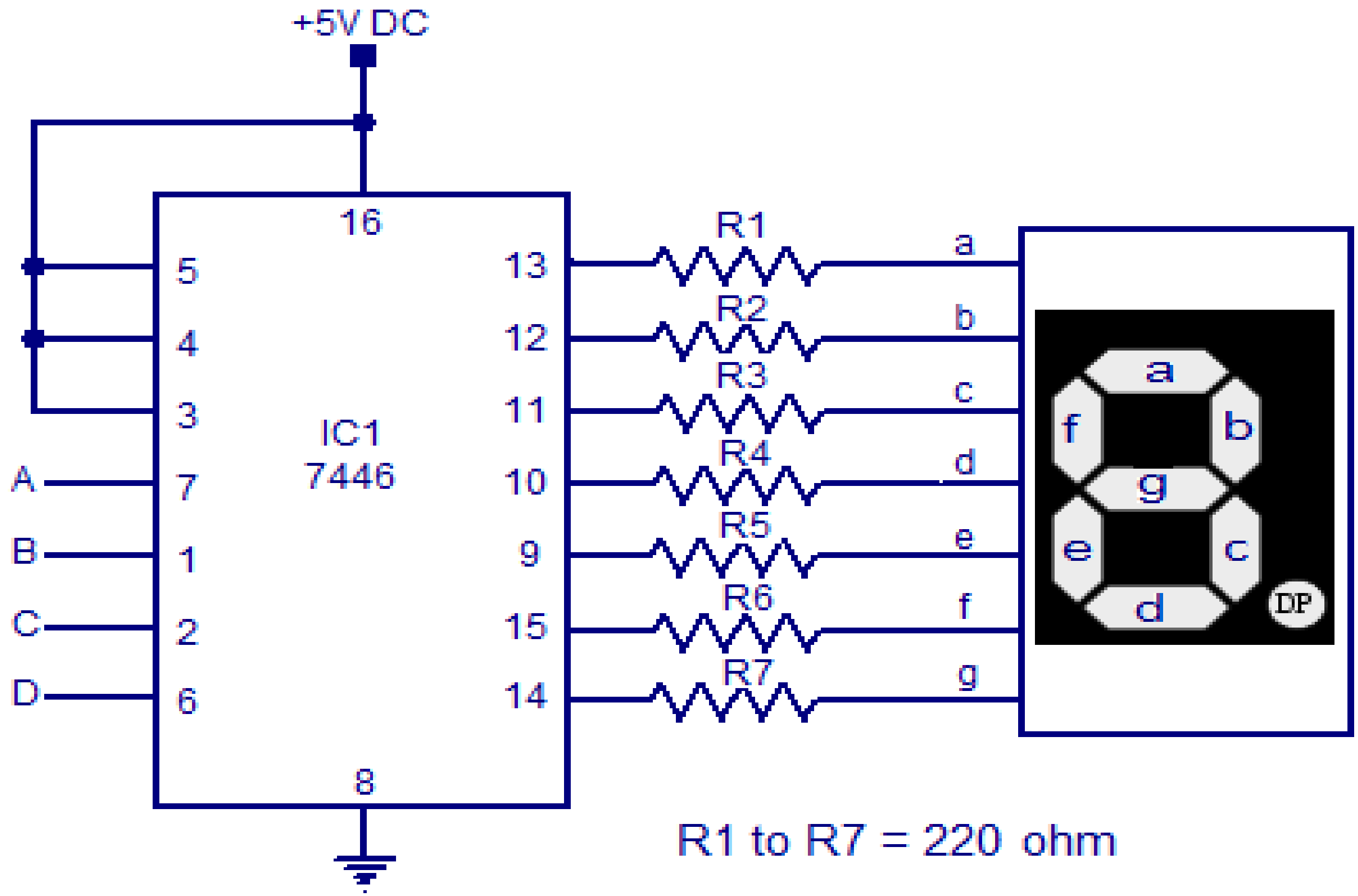
Boolean equations of 3-8 decoder

- **Implementation –**
  D0 is high when X = 0, Y = 0 and Z = 0. Hence,
- D0 = X' Y' Z'
- Similarly,
- D1 = X' Y' Z
- D2 = X' Y Z'
- D3 = X' Y Z
- D4 = X Y' Z'
- D5 = X Y' Z
- D6 = X Y Z'
- D7 = X Y Z

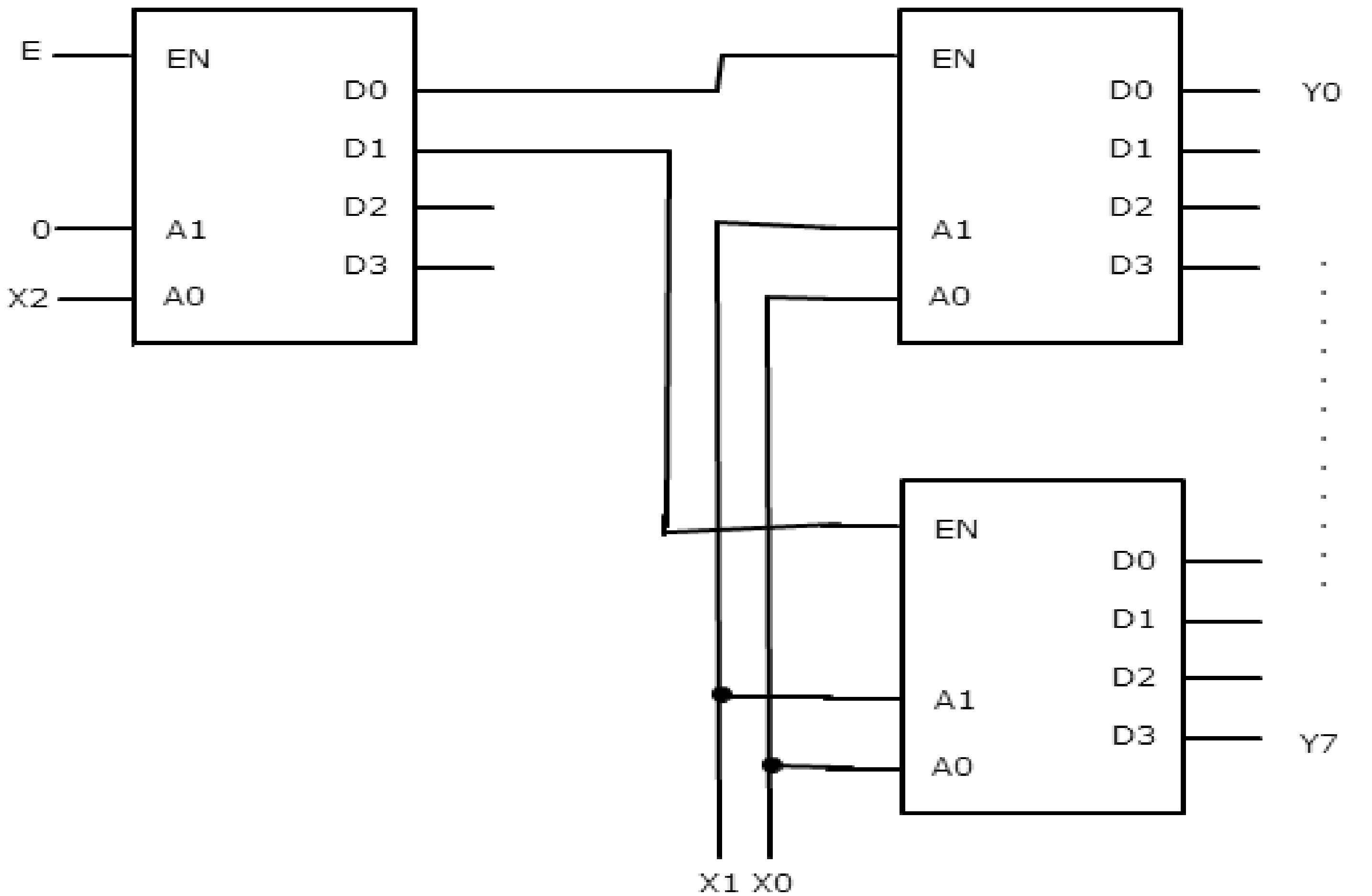

Decoder Networks
2-to-4
Decoder
Enable
Enable


A
B
C
D
A
DECODER
2 × 4
B
00
01
10
11
A
DECODER
2 × 4
B
00
01
10
11
0
1
2
3
4
5
6
7
8
9
A
B
C
D
E
F

BCD – 7 segment decoder



3-8 decoder using three 2-4 decoders

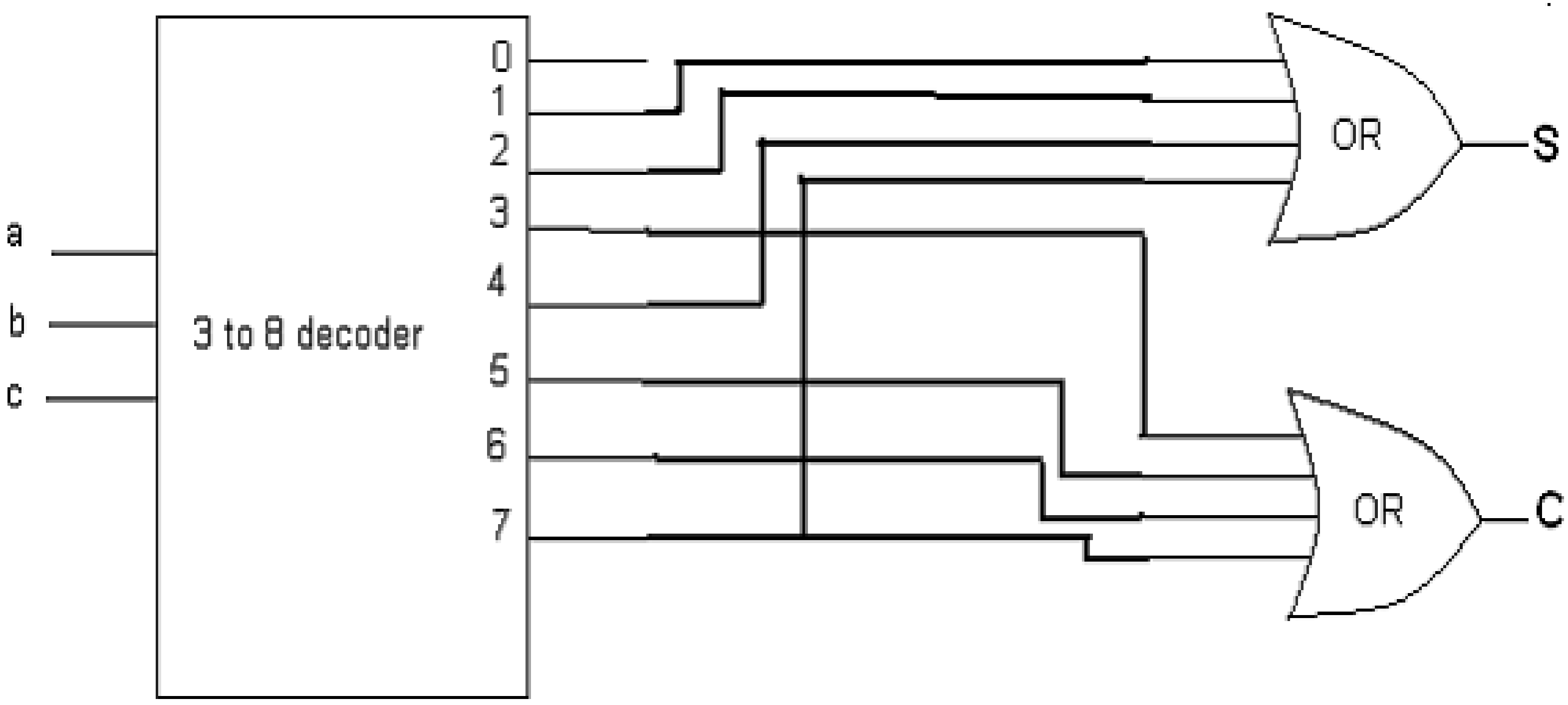


Example using Decoder

Realize the following functions using decoder.

$f_1 = \sum m(1,2.4,7)$

$f_2 = \sum m(3,5,6,7)$



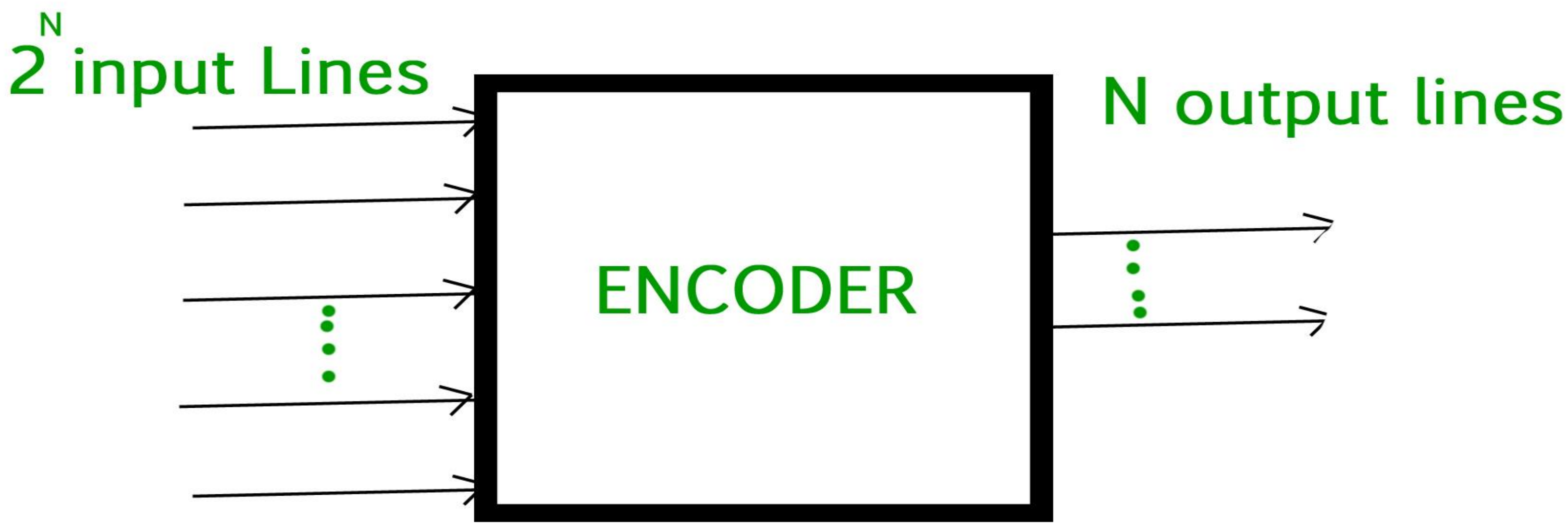
❖ What is an "Encoder"

- An **encoder** is a device or a combinational circuit that converts information from one format or code to another.
- Digital Encoder are more commonly called a **Binary Encoder** takes all its data inputs one at a time and then converts them into a single encoded output.
- An "n-bit" binary encoder has 2^n input lines and n-bit output lines

Encoders

Encoders –

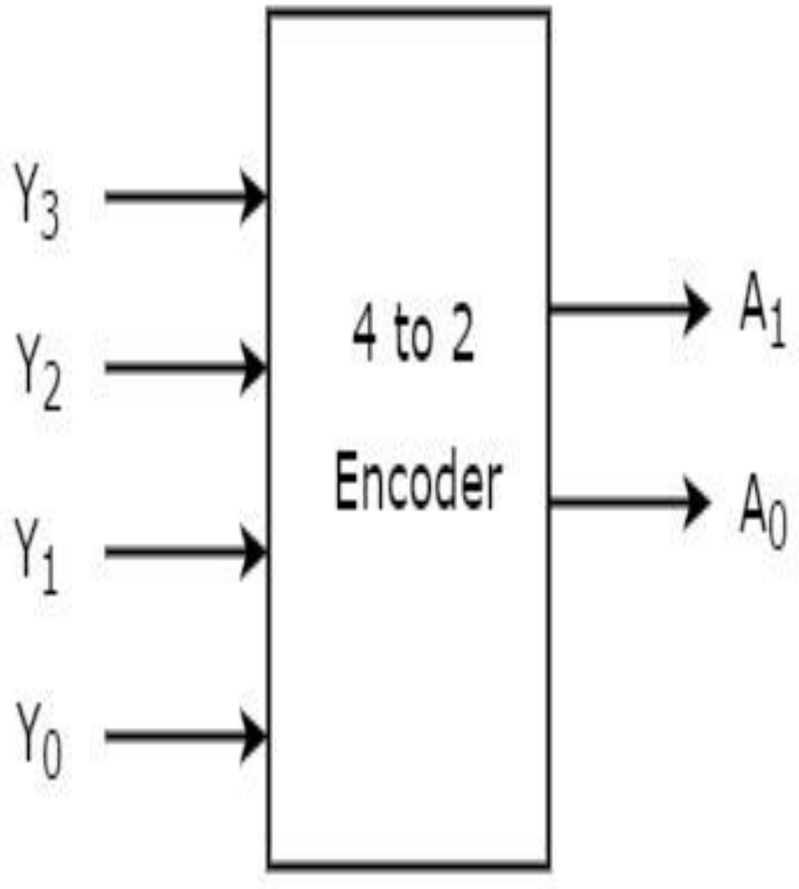
An encoder is a combinational circuit that converts binary information in the form of a 2^N input lines into N output lines, which represent N bit code for the input. For simple encoders, it is assumed that only one input line is active at a time.



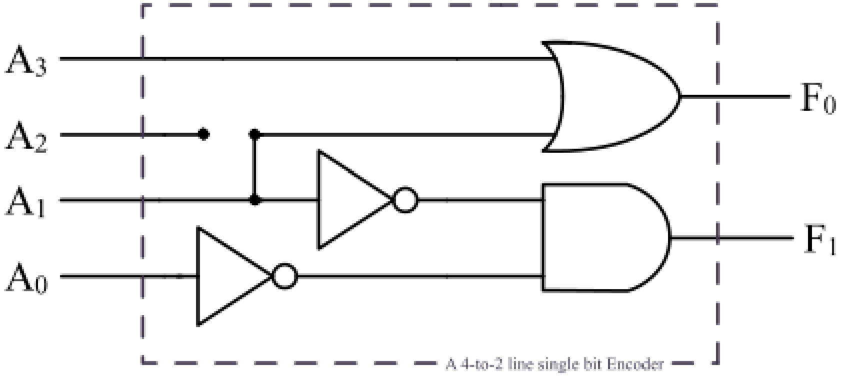
Types of Encoders

- Octal to binary encoder
- Hexadecimal to binary encoder
- Decimal to BCD encoder
- Priority encoder

4:2 Encoder



4-2 Encoder gate level implementation



Truth Table

A_3	A_2	A_1	A_0	F_1	F_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

Truth Table (Don't-Care-Conditions)

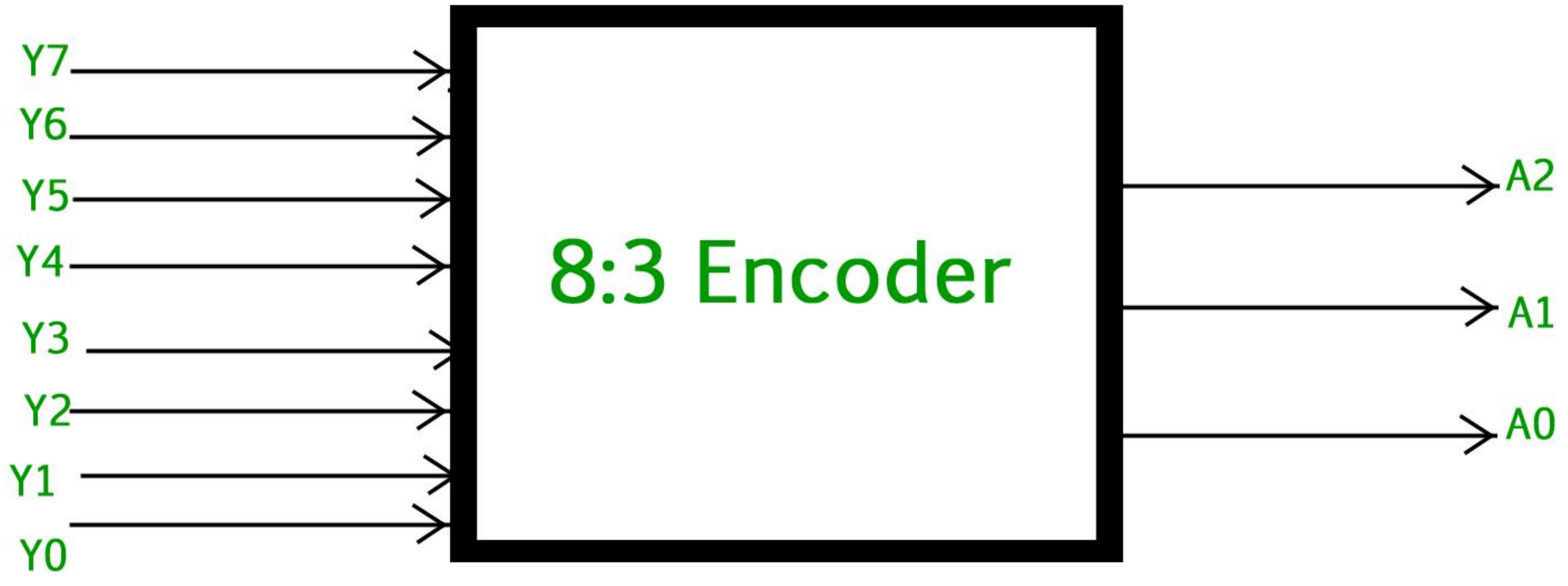
A_3	A_2	A_1	A_0	F_1	F_0
0	0	0	0	x	x
0	0	1	1	x	x
0	1	0	1	x	x
0	1	1	0	x	x
0	1	1	1	x	x
1	0	0	1	x	x
1	0	1	0	x	x
1	0	1	1	x	x
1	1	0	0	x	x
1	1	0	1	x	x
1	1	1	0	x	x
1	1	1	1	x	x

Minterm Equations

$$F_0 = A_3 + A_0$$

$$F_1 = \overline{A_1} \cdot \overline{A_0}$$

8:3 Encoder schematic



Octal to Binary encoder

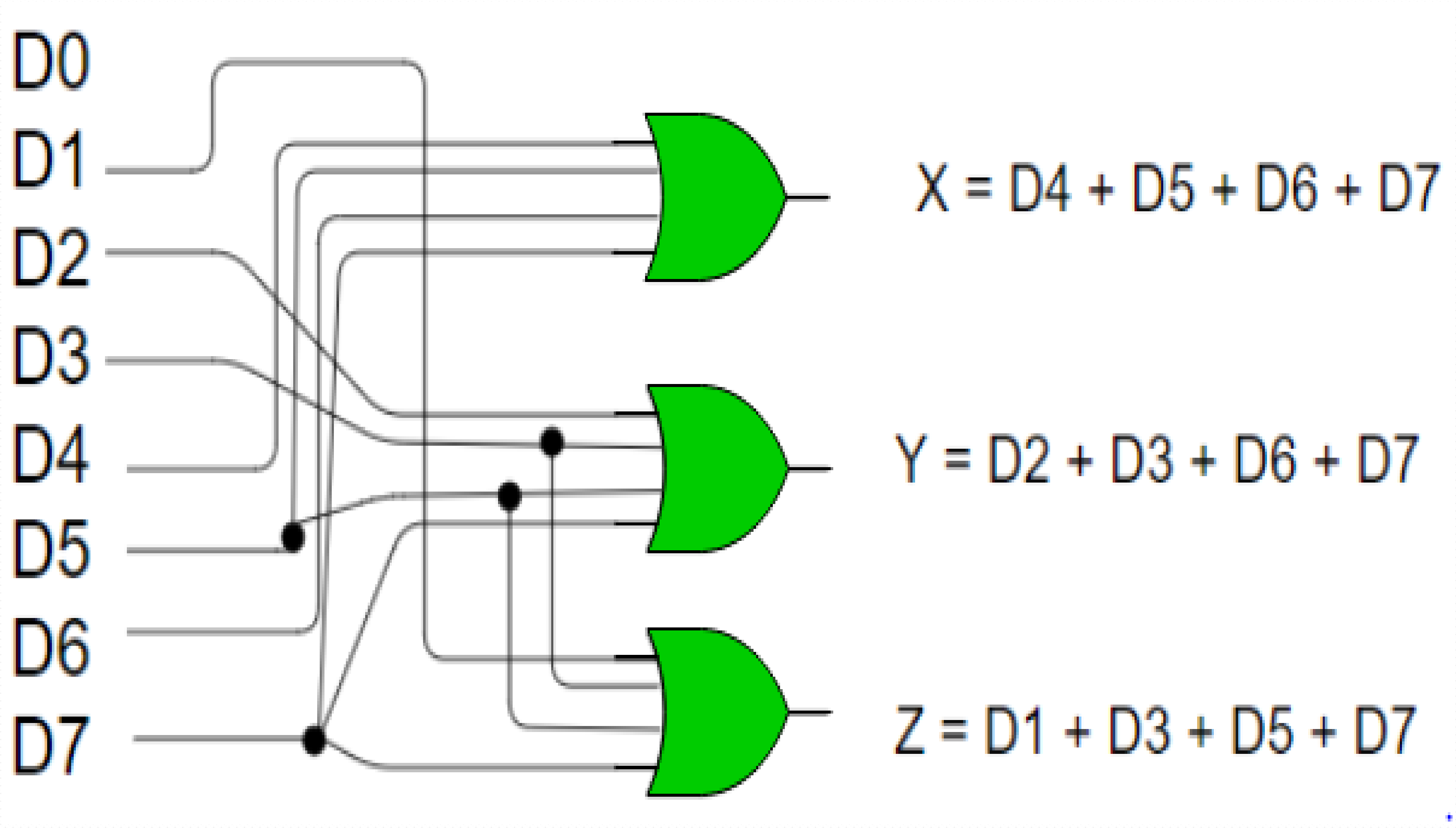
- **Truth table of 8 to 3 priority encoder:**

Inputs								Outputs		
D7	D6	D5	D4	D3	D2	D1	D0	X	Y	Z
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	X	0	0	1
0	0	0	0	0	1	X	X	0	1	0
0	0	0	0	1	X	X	X	0	1	1
0	0	0	1	X	X	X	X	1	0	0
0	0	1	X	X	X	X	X	1	0	1
0	1	X	X	X	X	X	X	1	1	0
1	X	X	X	X	X	X	X	1	1	1

Octal to Binary encoder

- As seen from the truth table, the output is 000 when D0 is active; 001 when D1 is active; 010 when D2 is active and so on.
- **Implementation –**
  From the truth table, the output line Z is active when the input octal digit is 1, 3, 5 or 7. Similarly, Y is 1 when input octal digit is 2, 3, 6 or 7 and X is 1 for input octal digits 4, 5, 6 or 7. Hence, the Boolean functions would be:
- $X = D4 + D5 + D6 + D7$
- $Y = D2 + D3 + D6 + D7$
- $Z = D1 + D3 + D5 + D7$

Octal to Binary encoder realization



Octal to Binary encoder limitations

- One limitation of this encoder is that only one input can be active at any given time.
- If more than one inputs are active, then the output is undefined. For example, if D6 and D3 are both active, then, our output would be 111 which is the output for D7.
- To overcome this, we use Priority Encoders
- Another ambiguity arises when all inputs are 0. In this case, encoder outputs 000 which actually is the output for D0 active. In order to avoid this, an extra bit can be added to the output, called the valid bit which is 0 when all inputs are 0 and 1 otherwise.

Priority Encoder

- A priority encoder is an encoder circuit in which inputs are given priorities. When more than one inputs are active at the same time, the input with higher priority takes precedence and the output corresponding to that is generated.
- Let us consider the 4 to 2 priority encoder as an example. From the truth table, we see that when all inputs are 0, our V bit or the valid bit is zero and outputs are not used. The x's in the table show the don't care condition, i.e, it may either be 0 or 1. Here, D3 has highest priority, therefore, whatever be the other inputs, when D3 is high, output has to be 11. And Do has the lowest priority, therefore the output would be 00 only when Do is high and the other input lines are low. Similarly, D2 has higher priority over D1 and Do but lower than D3 therefore the output would be 010 only when D2 is high and D3 are low (Do & D1 are don't care).

Truth Table of 4-2 Priority Encoder

Inputs				Outputs		
D_0	D_1	D_2	D_3	Y_1	Y_0	V
0	0	0	0	×	×	0
1	0	0	0	0	0	1
×	1	0	0	0	1	1
×	×	1	0	1	0	1
×	×	×	1	1	1	1

Implementation

- It can clearly be seen that the condition for valid bit to be 1 is that at least any one of the inputs should be high. Hence,
- V = D0 + D1 + D2 + D3
- For Y0:

Y0 = D2 + D3

D3 D2 \ D1 D0	00	01	10	11
00	x			
01	1	1	1	1
10	1	1	1	1
11	1	1	1	1

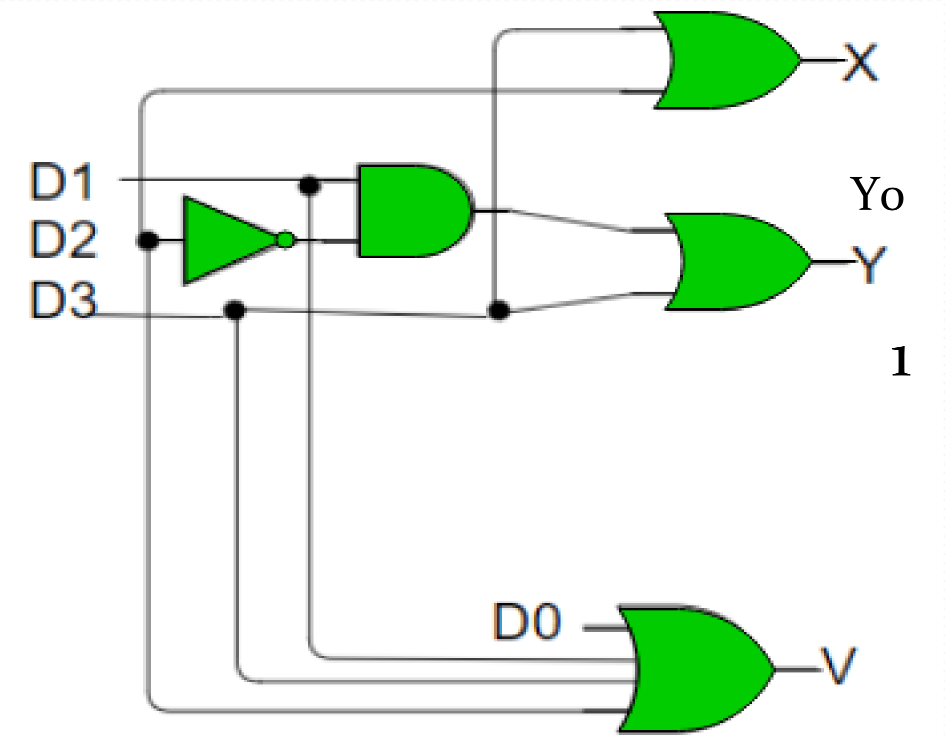
For Y1:

$Y_1 = D_1 D_2' + D_3$

D3 D2 \ D1 D0	00	01	10	11
00	x		1	1
01				
10	1	1	1	1
11	1	1	1	1

4-2 priority encoder realization

- Hence, the priority 4-to-2 encoder can be realized as follows:



What is Digital Comparator?

- As data comparison is mostly required in many digital systems at the time of logical or arithmetic functions, digital comparators are the one best option to compare data. Digital comparators are the most appropriate combinational logic circuits used to compare relative magnitudes of two binary numbers.
- The device accepts two binary numbers (A and B)as input and generates an output based on the magnitude of given inputs (example: A=B or A>B or A<B). Digital Comparators are developed through logic gates like AND, NOT or NOR gates. Digital comparators are available as identity comparators and magnitude comparators.

Magnitude Comparator

- Magnitude comparators are mostly utilized in microcontrollers and CPUs to address data comparison, register and perform all other arithmetic operations. Magnitude comparators are implemented in many devices and every auto-turn-off device is surely designed using a comparator.

- A comparator is a decision-making tool and it holds the ability to be executed in numerous control devices. Accepting two binary numbers as input (A and B), data comparison through magnitude comparators produces the output to indicate equality (A=B), logic 1 in two conditions when (A>B or A<B).

Types of Magnitude Comparators

- One bit
- Tow Bits
- Three Bits
- Four Bits

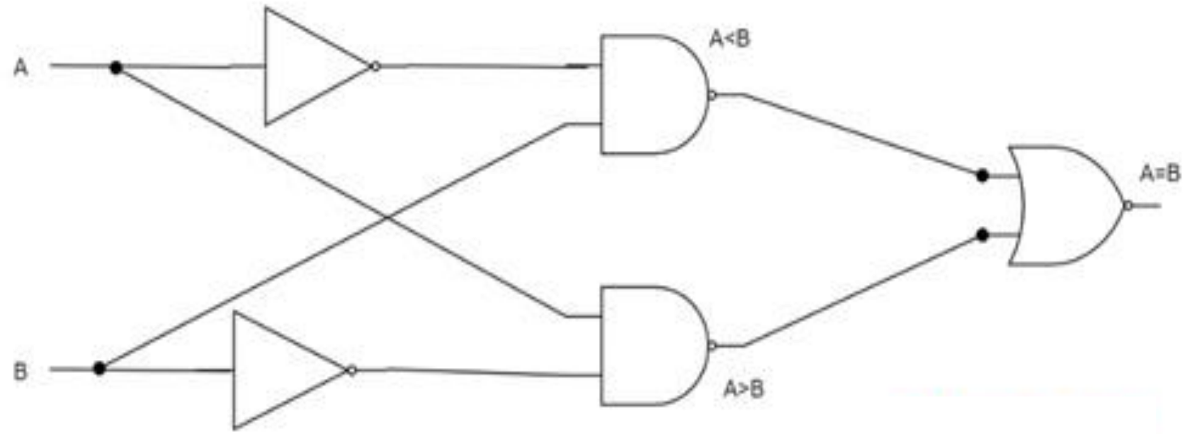
One bit magnitude comparator

- A comparator that compares two binary bits and produces three outputs based on the relative magnitudes of given binary bits is called a 1-bit magnitude comparator.
-

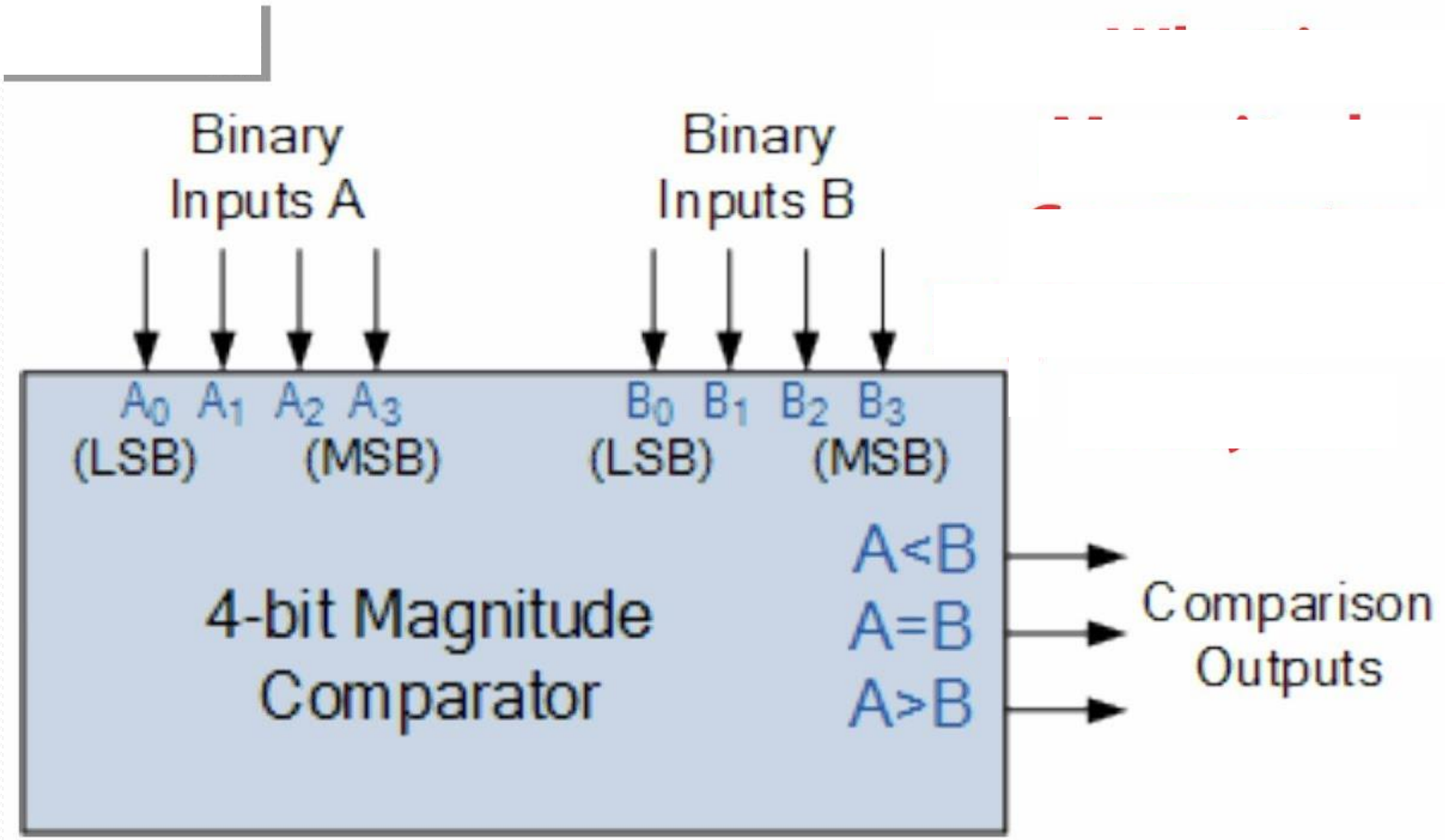
A	B	A<B	A>B	A=B
0	0	0	0	1
0	1	1	0	0
1	0	0	1	0
1	1	0	0	1

1 bit magnitude comparator implementation

- The truth table derives the expressions of A<B, A>B and A=B as below
- **A<B –> A’B**
- **A>B –> AB’**
- **A=B –> A’B’+AB**
- With these expressions, the Circuit diagram can be as follows



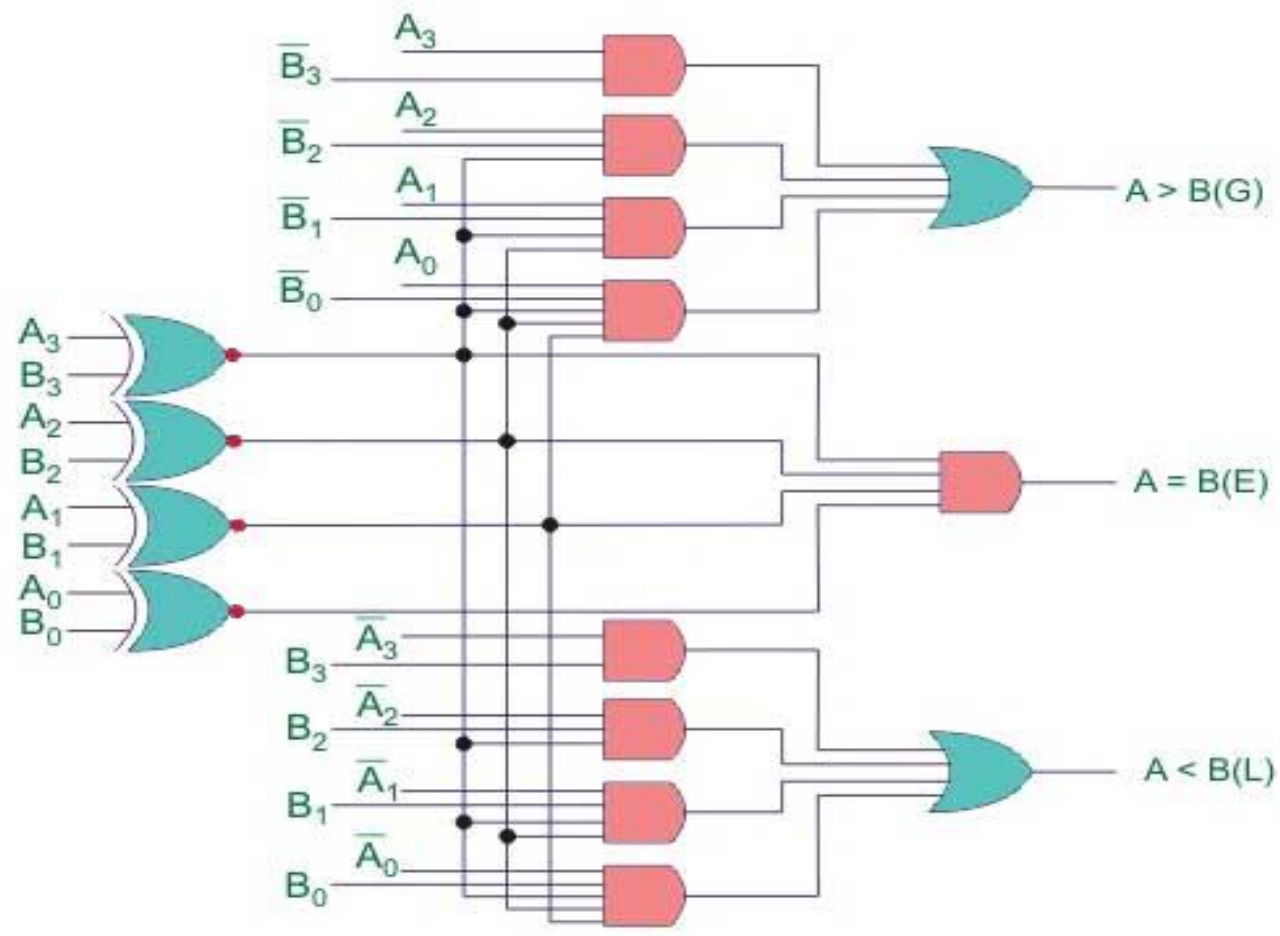
4bit magnitude comparator



4 bit magnitude comparator truth table

Truth table of 4-Bit Comparator						
COMPARING INPUTS				OUTPUT		
A3, B3	A2, B2	A1, B1	A0, B0	A > B	A < B	A = B
A3 > B3	X	X	X	H	L	L
A3 < B3	X	X	X	L	H	L
A3 = B3	A2 >B2	X	X	H	L	L
A3 = B3	A2 < B2	X	X	L	H	L
A3 = B3	A2 = B2	A1 > B1	X	H	L	L
A3 = B3	A2 = B2	A1 < B1	X	L	H	L
A3 = B3	A2 = B2	A1 = B1	A0 > B0	H	L	L
A3 = B3	A2 = B2	A1 = B1	A0 < B0	L	H	L
A3 = B3	A2 = B2	A1 = B1	A0 = B0	H	L	L
A3 = B3	A2 = B2	A1 = B1	A0 = B0	L	H	L
A3 = B3	A2 = B2	A1 = B1	A0 = B0	L	L	H
H = High Voltage Level, L = Low Voltage, Level, X = Don't Care						

4 bit magnitude comparator logic diagram



4bit magnitude comparator IC 7485